

EvalMem: An Operation-Level Diagnostic Framework for Long-Term Memory Systems

Zeyu Liu^{1,*} Jian Zhong^{1,*} Rongduo Han¹ Ziyang Wu¹ Shunye Tang¹
 Chenghao He¹ Yaxuan Yang¹ Yihang Qiu¹ Ailing Wang² Xiao Liang²
 Guohuan Xie³ Xiaokang Xue⁴ Gongchen Li¹ Haining Zhang^{1,†} Wei Wang^{1,†}

¹Nankai University ²vivo AI Lab

³Tsinghua University ⁴Zhejiang University

*Equal contribution; †Corresponding authors; Code: github.com/ZeyuuLiu/EvalMem

Abstract

Long-horizon interactions with LLM-based assistants require memory systems that preserve and update user states, preferences, and interaction histories. Existing evaluations report end-to-end QA accuracy and cannot determine whether errors arise from encoding, retrieval, or generation. We introduce **EvalMem**, an operation-level diagnostic framework with three parallel Examiners. For each query, the Encoding Examiner checks whether the target fact is stored, the Retrieval Examiner assesses whether the native retriever returns usable evidence, and the Generation Examiner tests whether the model can answer from oracle evidence. Their outputs form fine-grained multi-label defect codes. To improve store-level diagnosis, we adapt agentic RAG with a recall-first strategy that searches using both the query and source evidence, increasing recall of present evidence on LOCOMO from 70.2% to 95.6%. Evaluations of seven memory systems on LOCOMO, LONGMEMEVAL-S, and dynamic DynaMem-Bench identify retrieval as the most frequently attributed failure layer; in default LOCOMO, retrieval defects reach 22.1%, compared with 7.7% for encoding and 6.5% for generation. Guided by this diagnosis, **MemWiki**, a search-friendly auxiliary structure built from each system’s memory export, improves mean accuracy by 2.5 and 2.3 percentage points on LOCOMO and LONGMEMEVAL-S, respectively.

1 Introduction

Long-horizon interaction is becoming a central setting for LLM-based assistants, where personal assistants, tutors, health-support tools, and companion agents must maintain user states, preferences, plans, and past events across sessions (Park et al., 2023; Packer et al., 2023; Wang et al., 2023). Long-term memory systems support this capability by adding external storage and retrieval layers on top of LLMs (Li et al., 2025; Kang et al., 2025; Wang

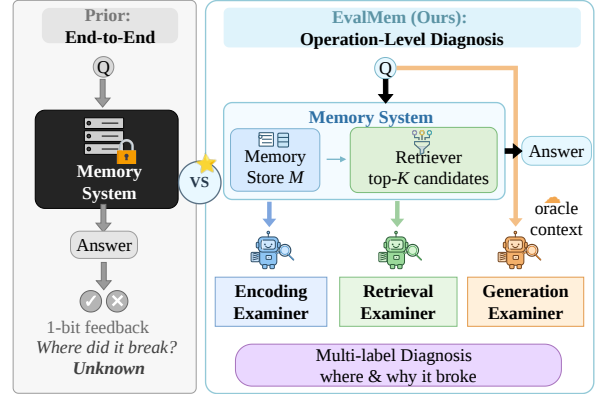


Figure 1: EvalMem at a glance. A query is examined by three parallel Examiners over Encoding, Retrieval, and Generation; their decisions are reconciled into a multi-label defect set instead of a single end-to-end score.

et al., 2025). Although their implementations vary widely, from hierarchical or temporal summaries to operating-system-style memory management, modular memory blocks, user profiles, and agentially constructed records (Kang et al., 2025; Li et al., 2026, 2025; Hu et al., 2026a; Tao et al., 2026; Wang et al., 2025; Yan et al., 2025; Xu et al., 2026), they share a common loop: encode past interactions, retrieve relevant records for a new query, and generate an answer from retrieved evidence. This shared loop makes unified attribution-oriented evaluation possible, yet current evaluation remains less informative than memory-system design demands (Ding et al., 2026).

Existing benchmarks have expanded long-term memory evaluation along several directions (Liu et al., 2024a). Static long-history benchmarks such as LOCOMO and LONGMEMEVAL test factual recall, temporal reasoning, multi-hop reasoning, knowledge updates, and abstention (Maharana et al., 2024; Wu et al., 2024); capability-oriented benchmarks examine hallucination, continual learning, memory operations, and broader memory competencies (Chen et al., 2025; Ai et al., 2025; Wang et al., 2026); and interactive benchmarks such as

AMEMGYM argue for on-policy evaluation, where the assistant participates in the interaction rather than being tested only on a fixed dialogue tape (Jiayang et al., 2026). Despite this progress, most protocols still report end-to-end answer correctness: a wrong answer does not reveal whether the relevant fact was not stored, not retrieved, or not used (Es et al., 2024; Lyu et al., 2025). Many datasets also rely on fixed histories, making it difficult to reflect how an assistant’s own dialogue choices shape memory-writing trajectory (Jiayang et al., 2026; Liu et al., 2024b). Failures caused by state evolution, stale records, or retrieval over the system’s actual memory store thus remain difficult to diagnose.

Building a diagnostic evaluation framework is challenging. It must abstract over heterogeneous storage formats, retrieval mechanisms, and generation pipelines; allow coupled failures across memory construction, retrieval, and answer generation; and reliably observe the memory store. If the evaluator fails to find a fact that was actually stored, it may incorrectly attribute a retrieval error to memory construction (Meng et al., 2022; Manakul et al., 2023).

To address these challenges, we propose EvalMem, an operation-level diagnostic framework summarized in Figure 1. EvalMem treats heterogeneous memory systems as an encoding–retrieval–generation pipeline and runs three independent Examiners in parallel for each query. The Encoding Examiner checks whether the target fact is present in the memory store, the Retrieval Examiner inspects whether the system’s native retriever returns usable evidence, and the Generation Examiner tests whether the model can answer from oracle evidence. An Attribution Agent merges their decisions into a fine-grained multi-label defect set, allowing co-occurring failures while suppressing retrieval defects when the source fact is absent from the store. To make the Encoding Examiner reliable when diagnostic retrieval is imperfect, we adapt agentic RAG with a recall-first principle: the probe searches the original memory store using both the query and its corresponding source evidence, reducing observation misses from 29.8% to 4.4% on LOCOMO samples with present evidence. We further construct DynaMem-Bench for on-policy dynamic evaluation and introduce MemWiki, a search-friendly auxiliary structure built from each system’s own memory export, to test whether diagnosis can guide system improvement. Our contri-

butions are threefold:

(1) Fine-grained attribution for memory-system evaluation. We introduce EvalMem, which uses Encoding, Retrieval, and Generation Examiners to produce an 11-code multi-label diagnosis.

(2) Reliable and dynamic diagnostic evaluation. We adapt agentic RAG into a bounded recall-first diagnostic routine and construct DynaMem-Bench for on-policy state evolution.

(3) Retrieval as the most frequently attributed failure layer. Across seven systems and three datasets, retrieval defects consistently exceed encoding and generation defects (22.1% vs. 7.7%/6.5% on LOCOMO, 18.3% vs. 8.5%/5.4% on LONGMEMEVAL-S, and 25.9% vs. 9.7%/5.0% on DynaMem-Bench); guided by this diagnosis, MemWiki improves mean accuracy by +2.5 pp on LOCOMO and +2.3 pp on LONGMEMEVAL-S.

2 EvalMem

EvalMem is an operation-level diagnostic framework for long-term memory systems. As shown in Figure 2, a tested system first runs natively: given dialogue history H and query Q , it builds or updates memory store $\mathcal{M}$, retrieves C_{original} , and generates $\hat{A}$. EvalMem then diagnoses this completed run with benchmark-side annotations: gold answer A_{gold} , task type τ , oracle evidence C_{oracle} , and key facts F_{key} distilled from that evidence. We first define task types and inputs, then present the three Examiners and Attribution Agent, the recall-first agentic-RAG routine used inside Encoding diagnosis, DynaMem-Bench for on-policy dynamic evaluation, and MemWiki as a diagnosis-guided actionability check.

2.1 Task Types and Diagnostic Inputs

Attribution is task-dependent, so we distinguish POS and NEG queries. A **POS** question is answerable from the original dialogue, so the system should store the relevant fact, retrieve it, and use it to answer; it tests whether the system can *remember and retrieve*. A **NEG** question lacks sufficient supporting evidence, so the ideal behavior is refusal rather than a concrete answer; it tests whether the system can *avoid unsupported answering*.

Each evaluated query is represented as

$$x = (H, Q, A_{\text{gold}}, \tau, F_{\text{key}}, C_{\text{oracle}}).$$

H is the dialogue history available before Q . A_{gold} is the expected answer or refusal, and $\tau \in$

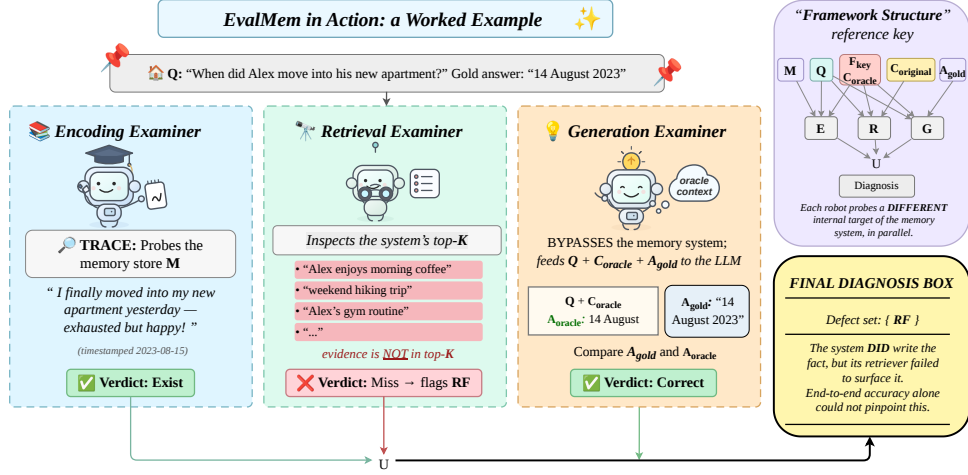


Figure 2: Overall view of EvalMem: the tested system runs natively, then Encoding, Retrieval, and Generation Examiners map task-aware states into a multi-label defect set.

$\{\text{POS}, \text{NEG}\}$ denotes the task type. C_{oracle} is the clean, complete evidence context. F_{key} is the set of atomic evidence facts required by Q : for POS it supports A_{gold} ; for NEG, no valid supporting fact exists, so it is empty or used only contrastively. After processing H , the tested system exposes $\mathcal{M}$; when asked Q , it returns C_{original} and $\hat{A}$. EvalMem can be integrated when a system exposes (i) an inspectable memory bank, (ii) the context actually passed to the generation model, and (iii) its final answer. The benchmark must provide, for each test question, the corresponding evidence text, i.e., the dialogues from the original conversational dataset.

2.2 Three Examiners and Attribution

EvalMem runs three Examiners in parallel. Each Examiner isolates one operation, returns a local state, and maps it to a task-aware defect subset. The Attribution Agent reconciles these subsets into a multi-label diagnosis while suppressing downstream defects that merely follow from upstream failures.

Encoding Examiner $\mathcal{E}_{\text{enc}}$. Inputs: $(Q, \tau, F_{\text{key}}, \mathcal{M})$. The Encoding Examiner asks whether the information required by Q is present in $\mathcal{M}$ under an oracle diagnostic view. This judgement is independent of the system's native retrieval result. Let P denote $\tau = \text{POS}$ and N denote $\tau = \text{NEG}$. All existential predicates range over $\mathcal{M}$. Predicates Supp , Ambig , and Wrong are evaluated against F_{key} and indicate full support, unresolved-reference support, and wrong-value support. $\text{Dirty}(m, Q)$ indicates an unsupported pseudo-fact relevant to a NEG query.

The encoding state is

$$S_{\text{enc}} = \begin{cases} \text{EXIST}, & P \wedge \exists m : \text{Supp}(m), \\ \text{DIRTY}, & N \wedge \exists m : \text{Dirty}(m, Q), \\ \text{CORRUPT}_{\text{Ambig}}, & P \wedge \exists m : \text{Ambig}(m), \\ \text{CORRUPT}_{\text{Wrong}}, & P \wedge \exists m : \text{Wrong}(m), \\ \text{MISS}, & \text{otherwise.} \end{cases} \quad (1)$$

For POS, EXIST means the key fact is stored correctly; MISS means no corresponding record is found; and the two CORRUPT states mean a related key is present but unusable. For NEG questions, MISS is the healthy state, while DIRTY means the store contains an unsupported pseudo-fact.

Retrieval Examiner $\mathcal{E}_{\text{ret}}$. Inputs: $(Q, \tau, F_{\text{key}}, C_{\text{original}})$. The Retrieval Examiner asks whether the native retrieval step returns the key information in an effective context. Write $C = C_{\text{original}}$ in the following equations. For POS questions, $F_{\text{key}} \preceq C$ means that all key facts are semantically supported by the retrieved context. For NEG, $\text{Mislead}_{\tau_H}(c, Q)$ marks a relevant-looking but insufficient, stale, or unsupported record with similarity above τ_H . The retrieval state is

$$S_{\text{ret}} = \begin{cases} \text{HIT}, & P \wedge F_{\text{key}} \preceq C, \\ \text{NOISE}, & N \wedge \exists c \in C : \text{Mislead}_{\tau_H}(c, Q), \\ \text{MISS}, & \text{otherwise.} \end{cases} \quad (2)$$

For POS, HIT means key information appears in the retrieved top- K context; MISS means it does not. For NEG, NOISE means retrieval returns a high-risk misleading record when no valid support exists. For POS hits, retrieval quality is further measured by the best evidence rank $r(F_{\text{key}})$ and the signal-to-noise ratio

$$\text{SNR}(C_{\text{original}}) = \frac{|\text{evid}(F_{\text{key}}) \cap C_{\text{original}}|_{\text{tok}}}{|C_{\text{original}}|_{\text{tok}}}. \quad (3)$$

A HIT is still retrieval-defective if the evidence is ranked too late ($r(F_{\text{key}}) > \tau_{\text{rank}}$) or diluted by noise ($\text{SNR} < \tau_{\text{snr}}$). Conversely, a POS MISS is counted as RF only when the Encoding Examiner confirms that the fact exists in $\mathcal{M}$; otherwise it is treated as upstream storage failure.

Generation Examiner $\mathcal{E}_{\text{gen}}$. Inputs: $(Q, \tau, C_{\text{oracle}})$. The Generation Examiner asks whether the answer model can solve the query when memory and retrieval are no longer bottlenecks. It feeds clean oracle context C_{oracle} and compares the oracle-path answer with A_{gold} : Let $\hat{A}_o = g(Q, C_{\text{oracle}})$ and $J = \text{Judge}$.

$$S_{\text{gen}} = \begin{cases} \text{PASS}, & J(\hat{A}_o, A_{\text{gold}}, \tau) = 1, \\ \text{FAIL}, & \text{otherwise.} \end{cases} \quad (4)$$

For POS questions, PASS means that the model can answer when the complete evidence is given; for NEG questions, it means that the model correctly refuses without support. When $S_{\text{gen}} = \text{FAIL}$, a subclassifier assigns the code: for POS questions, GF (Generation Faithfulness) when the model ignores the oracle evidence, and GRF (Generation Reasoning Failure) when the model reads the evidence but reasons incorrectly; for NEG questions, GH (Generation Hallucination) when the model fabricates an answer instead of refusing.

Multi-label union attribution. The final attribution is not a single root cause. For sample i , the Attribution Agent outputs

$$\mathcal{D}_{\text{total}}^{(i)} = \mathcal{D}_{\text{enc}} \cup \mathcal{D}_{\text{ret}} \cup \mathcal{D}_{\text{gen}}. \quad (5)$$

The healthy path is EXIST/HIT/PASS for POS and MISS/MISS/PASS for NEG. The Attribution Agent also applies an upstream masking rule: if a POS fact is absent from the store, retrieval MISS is not counted as RF; for NEG, NOISE is counted as NIR when the store is clean, while a retrieved pseudo-fact already written into the store is attributed to DMP. The 11 defect-code definitions are listed in Table 1 and Appendix B.4.

For reported defect statistics, we apply an *error gate*: only incorrectly answered samples contribute defect codes, and every rate takes the total number of questions in the benchmark as its denominator (the *full-query denominator*). Since attribution is multi-label, one incorrect sample may be assigned multiple defect codes.

Code	Full name	Layer	Condition
EM	Encoding Missing	Enc.	POS store MISS
EA	Encoding Ambiguous	Enc.	ambiguous value
EW	Encoding Wrong	Enc.	wrong value
DMP	Dirty Memory Pollution	Enc.	NEG dirty memory
RF	Retrieval Failure	Ret.	POS MISS with $S_{\text{enc}} = \text{EXIST}$
LATE	Retrieval Late	Ret.	hit rank $> \tau_{\text{rank}}$
NOI	Retrieval Noise	Ret.	hit SNR $< \tau_{\text{snr}}$
NIR	Noise-Induced Retrieval	Ret.	NEG misleading retrieval
GH	Generation Hallucination	Gen.	NEG fails to refuse
GF	Generation Faithfulness	Gen.	POS ignores oracle
GRF	Generation Reasoning Failure	Gen.	POS reasoning error

Table 1: The 11 defect codes used by EvalMem, grouped by layer. The full attribution for query i is the multi-label union $\mathcal{D}_{\text{total}}^{(i)}$.

Recall-first store diagnosis with adapted agentic RAG. The Encoding Examiner can only be trusted if it observes the memory store reliably. Weak evaluator-side search may mislabel a stored but hard-to-find fact as MISS, turning a retrieval problem into a false encoding defect. This risk is acute because the same evidence may appear as an atomic fact, dialogue turn, compressed summary, or document page.

To reduce such observation misses, EvalMem adapts agentic RAG into a read-only, recall-first diagnostic routine inside the Encoding Examiner. Unlike answer-oriented agentic RAG, this routine is used only for evidence-existence checking: it may use both Q and F_{key} to inspect $\mathcal{M}$, but it never modifies the store or supplies context to the tested generator.

The routine extracts anchors from (Q, F_{key}) , plans complementary views over the same memory export (original-query, temporal, entity, topical paraphrase, sparse keyword, and gap views), adapts retrieval budget to memory granularity, retrieves candidates with provenance metadata, and reranks them by direct support for F_{key} . Fusion uses reciprocal rank fusion,

$$s_{\text{rrf}}(d) = \sum_{v \in \mathcal{V}} \frac{1}{k_{\text{rrf}} + r_v(d)}, \quad (6)$$

because dense and sparse scores are not directly comparable. The reranker prioritizes candidates expressing the entity, relation, value, and temporal constraints in F_{key} .

Since the target fact may genuinely be absent, the routine uses bounded convergence rather than stopping when an answer seems sufficient:

$$\underbrace{|\text{pool}_t| \geq K_{\max}}_{\text{pool-full}} \vee \underbrace{\text{Jacc}(R_t, R_{t-1}) \geq \tau_J}_{\text{converged}} \vee \underbrace{t \geq T_{\max}}_{\text{max-iter}}. \quad (7)$$

The final high-recall pool is judged only by the Encoding Examiner.

2.3 DynaMem-Bench: On-Policy Dynamic Evaluation

DynaMem-Bench evaluates memory on trajectories produced by the tested system rather than fixed dialogue replay. Its construction keeps the semantic world and evaluation questions fixed for comparability while letting the actual dialogue unfold on policy for each system. The full construction pipeline is illustrated in Appendix C.9, Figure 8.

Offline blueprint. We define personas and user questions that depend on evolving personal states, then derive a state schema Σ and trajectory $\sigma_0, \dots, \sigma_T$. The trajectory specifies stable facts, updates, cascaded changes, and rollback cases, serving as the benchmark’s hidden world state. For each session t , we derive a must-expose set E_t : facts that must be naturally revealed because they first appear or change in σ_t . From the same trajectory, we generate session themes, grounded seed utterances, POS/NEG probes, gold answers, F_{key} , oracle contexts, and trap answers for stale or unsupported cases.

On-policy interaction. Each system interacts with the user simulator independently. Session t starts with the same seed utterance grounded in E_t for all systems; later user turns are role-played from the persona, current state σ_t , session theme, remaining unexposed items E_t^{rem} , dialogue history, and the system’s previous response. The assistant follows its native policy and updates memory normally. An exposure tracker removes mentioned facts, and a force-expose routine verbalizes any remaining required items near the final turn. Thus, systems produce different dialogues, but must-expose constraints guarantee the same observed fact set.

Read-only probes and validation. At the end of each session, scheduled evaluation questions are

asked through a read-only channel, so evaluation does not write into the memory store or affect later sessions. The probes include new POS questions about recently exposed facts, state-tracking POS questions whose answers change with the current state, and NEG questions about unmentioned, outdated, contradictory, or superficially similar but unsupported facts; all probes are judged by the same three Examiners. We validate the benchmark by checking that all required facts are exposed, that each question is answerable from its oracle evidence, that NEG subtypes and cascaded state changes behave as designed, and by human and model meta-evaluation. The resulting DynaMem-Bench has controlled state evolution, must-expose fairness constraints, read-only probes, and 962 validated questions over 10 personas.

2.4 Diagnosis-Guided Actionability Check with MemWiki

MemWiki is not part of the core scoring protocol. It checks whether retrieval defects identified by EvalMem are actionable: do the same stored facts become more useful when their retrieval surface is reorganized? MemWiki builds a search-friendly auxiliary index from each system’s memory export without oracle context, gold answers, F_{key} , evidence labels, or replacement of the native retriever.

Auxiliary records from raw memory.

MemWiki organizes the exported memory $\mathcal{M}$ into a wiki-style structure. Original records become source pages; repeated entities, topics, and events are aggregated into hub pages; and dynamic facts are versioned rather than overwritten. The structure is flattened into auxiliary set $\mathcal{A}$ with four retrieval-oriented views: **factual**, **tag**, **query-shaped**, and **aggregate** records, each pointing back to source records.

Integration with native retrieval. At query time, the tested system retrieves over the union of its original memory records and auxiliary records. Let $\text{Retr}(Q, \cdot)$ denote the native retriever, and let $\text{src}(a)$ return the original records pointed to by auxiliary record a . The final candidate pool is

$$C_{\text{final}} = \text{Rerank}\left(\text{Retr}(Q, \mathcal{M} \cup \mathcal{A}) \cup \bigcup_{a \in H_{\mathcal{A}}} \text{src}(a)\right), \quad (8)$$

where $H_{\mathcal{A}}$ is the subset of retrieved candidates that are auxiliary records. This implements storage-

surface injection, source expansion, and reranking while keeping the original evidence and native retriever fixed during this retrieval-side intervention.

3 Experiments

We evaluate EvalMem on three datasets (LOCOMO (Maharana et al., 2024), LONGMEMEVAL-S (Wu et al., 2024), and DynaMem-Bench) under three LLM backbone settings (GPT-4O-MINI, GPT-4.1, and QWEN3-30B-INSTRUCT (Yang et al., 2025)). §3.1 describes the experimental setup. §3.2 reports end-to-end accuracy together with operation-level defect rates, including cross-dataset and backbone comparisons. §3.3 further examines whether the diagnostic components are reliable and actionable through three mechanism studies: agentic-RAG reliability, the MemWiki intervention, and top- K sensitivity.

3.1 Setup

Datasets. LOCOMO (Maharana et al., 2024) contains 1986 QA pairs over 10 multi-session dialogues (1540 POS and 446 NEG). LONGMEMEVAL-S (Wu et al., 2024) contains 500 items (470 POS and 30 NEG) with approximately 115K-token haystacks across about 50 sessions. DynaMem-Bench (§2.3) contains 962 validated questions over 10 personas (790 POS and 172 NEG).

Memory systems. We evaluate seven released memory systems through dedicated adapters: TiMem (Li et al., 2026), O-Mem (Wang et al., 2025), EverMemOS (Hu et al., 2026a), MemOS (Li et al., 2025), MemBox (Tao et al., 2026), MemoryOS (Kang et al., 2025), and GAM (Yan et al., 2025). These systems cover hierarchical-temporal memory, system-level multi-tier memory management, engram-lifecycle modeling, memory-block abstraction, agentic deep-research memory, and user-profile-centric memory.

Models and hyper-parameters. Unless otherwise specified, both the memory-system-side LLM calls and answer generation use GPT-4O-MINI. GPT-4.1 and QWEN3-30B-INSTRUCT are used for the cross-backbone study. The EvalMem-side agents, including the three Examiners, use GPT-5-MINI. The user simulator for DynaMem-Bench uses GPT-4O. To keep retrieval comparisons controlled, we standardize the embedding encoder used by each system’s embedding-based

retrieval/index and by the corresponding diagnostic retrieval to Qwen3-Embedding-0.6B (Zhang et al., 2025) (1024-d). This replaces only the encoder (and rebuilds its vectors/index entries as needed); memory construction, index type/structure, retrieval logic, answer backbone, and EvalMem judges remain unchanged. Appendix C.2 reports an embedding-only rerun with each system’s released/-native encoder: systems whose encoder changes shift accuracy by 1.2–6.8 pp, while Retrieval remains the most frequently attributed layer; TiMem uses the standardized encoder natively. Temperatures are all fixed at $\tau = 0.0$ for answer generation and judging. Unless otherwise stated, retrieval uses Top- $K = 10$; we analyze this choice in §3.3 and Appendix C.8. We set $\tau_{\text{rank}} = 5$, $\tau_{\text{snr}} = 0.20$, and $\tau_H = 0.55$. These values were selected with system-stratified human calibration on fixed pilot samples; Appendix C.3 reports one-at-a-time sensitivity sweeps and calibration macro-F1, with each selected value fixed before aggregating test results. The agentic-RAG diagnostic procedure uses $T_{\text{max}} = 4$, pool cap 50, and Jaccard convergence threshold 0.85.

3.2 End-to-End Results across Datasets and Backbones

Table 2 reports per-system end-to-end accuracy, the three layer-aggregate defect rates, and the full 11-code distribution for three main configurations: LOCOMO with the default backbone, LOCOMO with GPT-4.1, and DynaMem-Bench with the default backbone. All code and layer rates follow the definition in §2.2: unless otherwise noted, only incorrectly answered samples contribute defect codes, the denominator is the total number of questions in the corresponding benchmark, and one sample may be assigned multiple defect codes. Appendix C.1 provides the complete $7 \times 3 \times 3$ matrix over seven systems, three datasets, and three backbones. We use LONGMEMEVAL-S as an auxiliary generalization set. Because this dataset is relatively small and contains only 30 NEG samples, we report only accuracy and layer-level aggregate defect rates for it.

Finding 1: retrieval is the most frequently attributed failure layer. Under the default backbone, the mean retrieval-defect incidence is consistently higher than both encoding and generation across all three datasets: 22.1 vs. 7.7/6.5 on LOCOMO, 18.3 vs. 8.5/5.4 on LONGMEMEVAL-

System	Encoding ↓					Retrieval ↓					Generation ↓				Acc ↑
	EM	EA	EW	DMP	Enc	RF	LATE	NOI	NIR	Ret	GF	GRF	GH	Gen	
Set 1: LoCoMo + GPT-4O-MINI						N = 1986		mean Acc = 68.1, mean Ret = 22.1							
TiMem	3.3	1.8	1.2	0.8	7.1	9.1	2.7	3.5	1.7	17.0	2.1	2.6	1.3	6.0	75.3
O-Mem	2.1	1.3	1.0	0.5	4.9	10.5	3.3	3.7	2.1	19.6	2.4	2.5	1.2	6.1	74.9
EverMemOS	4.6	2.0	1.3	0.7	8.6	10.4	3.4	4.0	2.2	20.0	2.5	2.4	1.4	6.3	72.4
MemOS	4.2	1.6	1.4	0.9	8.1	11.0	3.6	4.4	2.0	21.0	2.6	2.7	1.2	6.5	70.1
MemBox	2.4	1.4	1.1	1.0	5.9	14.7	4.5	4.3	2.2	25.7	2.7	2.6	1.6	6.9	63.2
MemoryOS	4.7	2.3	1.5	0.8	9.3	13.7	4.9	5.0	2.4	26.0	2.3	2.8	1.5	6.6	60.8
GAM	5.2	2.5	1.3	1.1	10.1	14.3	4.4	5.6	1.4	25.7	2.8	2.7	1.6	7.1	60.2
mean	3.8	1.8	1.3	0.8	7.7	12.0	3.8	4.4	2.0	22.1	2.5	2.6	1.4	6.5	68.1
Set 2: LoCoMo + GPT-4.1						N = 1986		mean Acc = 70.9, mean Ret = 20.2							
O-Mem	1.5	1.0	0.7	0.5	3.7	9.5	2.9	3.3	1.8	17.5	1.4	1.9	0.9	4.2	78.1
TiMem	2.5	1.4	1.0	0.6	5.5	8.1	2.6	3.1	1.6	15.4	1.5	1.8	0.8	4.1	77.8
EverMemOS	3.4	1.5	1.0	0.7	6.6	9.4	3.0	3.6	1.8	17.8	1.6	1.7	1.1	4.4	75.2
MemOS	3.0	1.5	1.1	0.5	6.1	10.0	3.2	3.8	1.7	18.7	1.8	2.0	0.9	4.7	73.1
MemBox	2.4	1.1	0.9	1.0	5.4	13.6	4.0	4.3	2.1	24.0	1.9	2.1	1.2	5.2	65.7
MemoryOS	3.7	1.8	1.2	0.8	7.5	13.1	4.3	4.6	2.4	24.4	1.7	2.0	1.1	4.8	63.8
GAM	4.5	2.0	1.2	0.9	8.6	13.0	4.2	5.1	1.5	23.8	2.0	2.2	1.0	5.2	62.8
mean	3.0	1.5	1.0	0.7	6.2	11.0	3.5	4.0	1.8	20.2	1.7	2.0	1.0	4.6	70.9
Set 3: DynaMem-Bench + GPT-4O-MINI						N = 962		mean Acc = 60.8, mean Ret = 25.9							
TiMem	3.5	2.0	1.4	0.7	7.6	9.3	3.3	3.8	2.6	19.0	1.5	1.9	0.9	4.3	71.2
O-Mem	3.1	1.8	1.7	0.9	7.5	10.9	3.6	4.2	2.8	21.5	1.7	1.8	1.0	4.5	68.9
MemOS	4.3	2.1	1.5	1.1	9.0	13.3	4.1	4.5	3.5	25.4	1.8	2.0	1.1	4.9	62.5
EverMemOS	4.8	2.2	1.6	1.0	9.6	13.7	4.0	4.7	3.7	26.1	1.6	2.1	1.0	4.7	61.0
MemoryOS	5.4	2.6	1.7	1.3	11.0	14.7	4.4	5.2	4.1	28.4	1.9	2.0	1.2	5.1	56.4
MemBox	4.5	2.3	1.8	1.4	10.0	15.4	4.6	5.0	4.1	29.1	2.0	2.2	1.3	5.5	55.9
GAM	6.5	2.9	2.1	1.5	13.0	16.9	5.1	5.6	4.4	32.0	2.1	2.4	1.4	5.9	49.6
mean	4.6	2.3	1.7	1.1	9.7	13.5	4.2	4.7	3.6	25.9	1.8	2.1	1.1	5.0	60.8

Table 2: Accuracy and per-layer defect rates for seven memory systems on three configurations: LoCoMo+GPT-4O-MINI (Set 1), LoCoMo+GPT-4.1 (Set 2, cross-backbone), and DynaMem-Bench +GPT-4O-MINI (Set 3). Columns: encoding codes + **Enc** total, retrieval codes + **Ret** total, generation codes + **Gen** total, end-to-end **Acc**; defect codes are assigned only to incorrectly answered outputs, while each defect column reports a sample-level union rate with the full-query denominator. Defect codes may overlap. **Bold red** = best per column within each Set; **blue underline** = second-best (ties marked at every tied cell). QWEN3, LONGMEMEVAL-S, and the full $7 \times 3 \times 3$ matrix are in Appendix C.1.

S, and 25.9 vs. 9.7/5.0 on DynaMem-Bench. On the two datasets with reliable fine-grained code breakdowns, the largest retrieval sub-code is RF, consistent with the diagnostic observation that the relevant fact is present in the memory store but is not surfaced by the system’s native retriever.

Finding 2: system rankings depend on dataset characteristics. No single memory system dominates across all evaluation settings. Under the default backbone, TiMem achieves the highest accuracy on LoCoMo and DynaMem-Bench, whereas EverMemOS leads on LONGMEMEVAL-S. Under GPT-4.1 on LoCoMo, O-Mem slightly exceeds TiMem by 0.3 pp. This pattern is consistent with the different emphases of the datasets: LONGMEMEVAL-S stresses factual recall from very long haystacks, whereas DynaMem-Bench emphasizes state evolution and stale-evidence handling. There-

fore, a single aggregate leaderboard can obscure different capability profiles, and memory-system evaluation should report layer-level diagnostics together with cross-dataset results.

Finding 3: dynamic state changes expose stale-evidence failures. Compared with LoCoMo under the same default backbone, all seven systems obtain lower accuracy on DynaMem-Bench, with a mean drop of -7.3 pp. The drop is mainly accompanied by higher retrieval and encoding defect rates ($+3.8$ and $+2.0$ pp), while generation defects decrease by 1.5 pp. The retrieval sub-code with the largest increase is NIR, which rises from 2.0% on LoCoMo to 3.6% on DynaMem-Bench. In the dynamic benchmark audit, the rollback subset is also the hardest aggregate subset, with only 47.3% accuracy (Appendix C.10). These results suggest that current systems can often retain old evidence after

state changes, but they do not reliably prevent stale or misleading records from affecting later answers.

Backbone effect. Replacing GPT-4O-MINI with GPT-4.1 increases mean accuracy by +2.8, +4.7, and +3.7 pp on LOCOMO, LONGMEMEVAL-S, and DynaMem-Bench, respectively. Using QWEN3-30B-INSTRUCT keeps mean accuracy within -1.3 pp of the default backbone. System rankings remain highly stable across backbone settings (Spearman $\rho \geq 0.929$), with rank flips mainly occurring between near-tied systems. Thus, the observed cross-system differences are not artifacts of a single backbone.

3.3 Mechanism Studies: Diagnostic Reliability, Intervention, and Top- K

Figure 3 summarizes the three mechanism studies. Full per-cell numbers and ablations are reported in Appendix C.6.

Agentic-RAG reliability (panel c). The first mechanism study evaluates whether the agentic-RAG procedure used inside the Encoding Examiner can make store-level observation more reliable. This study concerns the evaluator-side diagnosis of whether relevant evidence exists in the memory store, rather than the tested system’s native retrieval quality. On the LOCOMO POS subset, a vanilla diagnostic retriever using only Q misses 29.8% of evidence known to be present. Adding only the agentic-RAG pipeline lowers the MISS rate to 18.4%; adding only the evidence-aware query lowers it to 11.3%; using both further lowers it to 4.4% (Appendix C.6). A leave-one-out analysis from the full configuration shows that the evidence-aware query is the largest contributor: removing it raises MISS by 14.0 pp. The remaining agentic-RAG pipeline mechanisms contribute 0.7–2.5 pp each. These results support using the full diagnostic configuration to reduce false encoding attributions caused by incomplete evaluator-side search.

MemWiki intervention (panel d). MemWiki provides an actionability check for a diagnosis-guided retrieval-side intervention. On LOCOMO with GPT-4O-MINI, enabling all four auxiliary structures improves mean final accuracy by +2.5 pp and reduces the mean retrieval-defect rate by 3.2 pp. Gains are largest for systems with higher observed baseline retrieval-defect rates (GAM +4.6 pp; MemBox +4.2 pp) and smallest for TiMem (+0.3 pp), whose native profile layer

System	Bsl Acc	+MW Acc	Δ Acc	Δ Ret
EverMemOS	82.6	83.4	+0.8	−1.4
O-Mem	81.4	82.8	+1.4	−2.6
MemBox	77.0	79.6	+2.6	−3.4
GAM	76.6	79.4	+2.8	−3.6
TiMem	76.0	76.6	+0.6	−1.4
MemOS	65.0	68.4	+3.4	−4.4
MemoryOS	55.4	59.8	+4.4	−5.4
<i>mean</i>	73.4	75.7	+2.3	−3.2

Table 3: MemWiki on LONGMEMEVAL-S (gpt-4o-mini). Mean Acc / Ret shifts mirror the LOCOMO result within 0.2 pp; per-system significance and Pearson $r(\Delta\text{Ret}, \Delta\text{Acc})$ in Appendix C.7.

already supplies part of this surface. Repeating the intervention on LONGMEMEVAL-S yields +2.3 pp accuracy and a 3.2 pp retrieval-defect reduction (Table 3). McNemar tests and quota ablations are in Appendix C.7.

Top- K sensitivity (panels a, b). Increasing K monotonically improves POS Hit@ K , but it does not monotonically improve end-to-end accuracy. Hit@ K shows an elbow around $K = 10$, with marginal gains below 1.0 pp per additional candidate beyond that point. Final accuracy peaks at $K \in \{8, 10\}$ for the mean curve and for nearly all systems, then declines as larger contexts introduce more noise. Appendix C.8 shows the mechanism: from $K = 10$ to $K = 20$, RF decreases but NOI rises from 5.5% to 12.5%, indicating that additional candidates increasingly become distractors. We therefore use $K = 10$ as the retrieval budget.

Diagnostic stability and overhead. Holding the baseline systems’ LOCOMO outputs fixed and rerunning the full EvalMem diagnostic pipeline five times yields layer-level standard deviations of at most 0.25 pp and Fleiss’ $\kappa \geq 0.89$; this measures diagnostic stochasticity rather than memory-system seed variance (Appendix C.4). The mean per-example EvalMem diagnostic overhead is 8.48 calls, 47.2K input tokens, and 2.04K output tokens, with critical-path latency of 13.0/28.9 seconds at p50/p95. Because the three Examiners run in parallel before Attribution, the critical path is approximately $\max(t_E, t_R, t_G) + t_A$, not the sum of component latencies (Appendix C.5).

4 Conclusion

EvalMem turns long-term memory evaluation from end-to-end scoring into operation-level attribution. By combining Encoding, Retrieval, and Genera-

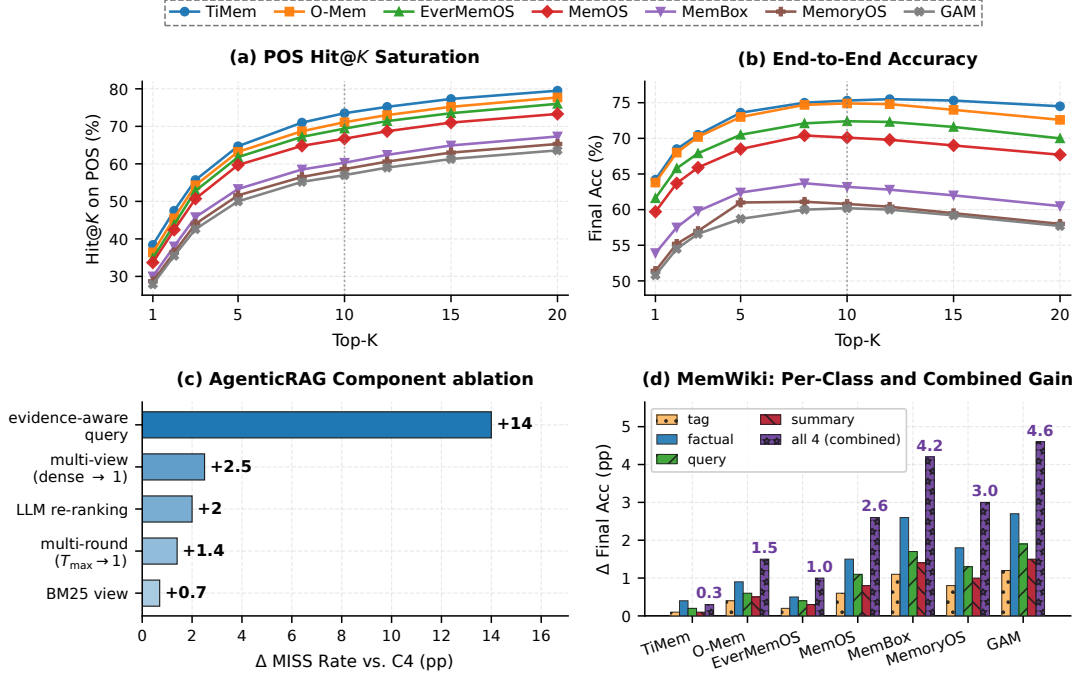


Figure 3: Mechanism analyses on LOCOMO with GPT-4O-MINI. **(a)** POS Hit@ K saturates around $K = 10$. **(b)** Final accuracy peaks at $K \in \{8, 10\}$. **(c)** Agentic-RAG reliability analysis: the evidence-aware query is the largest contributor to reducing diagnostic MISS, while the agentic-RAG pipeline mechanisms provide additional gains. **(d)** MemWiki per-class and combined intervention: factual records are the strongest single auxiliary class.

tion Examiners with task-aware multi-label rules, it diagnoses whether a failure arises because the relevant fact was not stored, not retrieved, or not used. A recall-first agentic-RAG diagnostic routine makes store-level observation more reliable, while DynaMem-Bench extends evaluation to on-policy state evolution. Across seven memory systems and three datasets, retrieval is the most frequently attributed failure layer in the evaluated settings: the reported Retrieval attribution frequency exceeds the Encoding and Generation frequencies in every setting, and dynamic interaction exposes stale-evidence failures that static replay can miss. Finally, MemWiki improves accuracy on LOCOMO and LONGMEMEVAL-S by reorganizing existing memories into a more searchable surface, providing an actionability check for retrieval-side reorganization.

Limitations

The three Examiners in EvalMem are realised as LLM-judge components, and the corresponding attributions are therefore subject to the standard caveats that apply to LLM-as-judge protocols; we report Cohen’s κ against trained annotators in Appendix D so that this dependency is quantified. Our experiments cover three datasets and three

answer-generation backbones drawn from a single language; broader coverage — additional languages and domains — would further consolidate the generality of the framework. We evaluate seven public memory systems under a controlled standardized configuration and an embedding-only native-encoder robustness check; how attribution behaves under system-specific tuning remains open for follow-up work.

Ethics Statement

All datasets used in this work are either publicly available (LOCOMO, LONGMEMEVAL-S) or generated through a controlled persona-driven simulator (DynaMem-Bench), and no human-subject data was collected during this study. The personas in DynaMem-Bench are synthetic and do not represent any real individual; they are derived from a small grid over commonly studied user-profile dimensions (age, education, communication style, change rate, NEG sensitivity) and are released alongside the benchmark to support reproducible evaluation. The memory systems we evaluate are released by their respective authors with open licences; we use them strictly within those licence terms and do not redistribute model weights. We anticipate the primary impact of EvalMem to be di-

agnostic: by making operation-level failure modes visible, the framework helps the community measure long-term memory progress more rigorously and avoid over-claiming based on opaque end-to-end metrics. We see no immediate dual-use risk specific to this work beyond the general considerations that apply to long-term memory deployments (privacy of accumulated user content, traceability of stored records); these considerations are inherited from the memory systems we evaluate rather than introduced by the framework itself.

Acknowledgments

This research is supported by National Natural Science Foundation of China under Grant 62406163, Tianjin Science and Technology Plan Project under Grant 25ZXSFSN00150, and the Fundamental Research Funds for the Central Universities under Grant 63261039. We thank the anonymous reviewers for their constructive feedback, which helped us clarify the scope, attribution rules, and limitations of this work. We also thank the developers and maintainers of the open-source memory systems and benchmark resources used in our experiments for making these resources available to the community.

References

- Qingyao Ai, Yichen Tang, Changyue Wang, Jianming Long, Weihang Su, and Yiqun Liu. 2025. Memory-bench: A benchmark for memory and continual learning in llm systems. *arXiv preprint arXiv:2510.17281*.
- Akari Asai, Zeqiu Wu, Yizhong Wang, Avi Sil, and Hannaneh Hajishirzi. 2024. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *International conference on learning representations*, volume 2024, pages 9112–9141.
- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, and 1 others. 2024. Long-bench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 3119–3137.
- Ding Chen, Simin Niu, Kehang Li, Peng Liu, Xiangping Zheng, Bo Tang, Xinchu Li, Feiyu Xiong, and Zhiyu Li. 2025. Halumem: Evaluating hallucinations in memory systems of agents. *arXiv preprint arXiv:2511.03506*.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshray Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.
- Yihang Ding, Wanke Xia, Yiting Zhao, Jinbo Su, Jialiang Yang, Zhengbo Zhang, Ke Wang, and Wenming Yang. 2026. Memground: Long-term memory evaluation kit for large language models in gamified scenarios. *arXiv preprint arXiv:2604.14158*.
- Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. Ragas: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th conference of the european chapter of the association for computational linguistics: system demonstrations*, pages 150–158.
- Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, and 1 others. 2026a. Ever-memos: A self-organizing memory operating system for structured long-horizon reasoning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 45836–45853.
- Yuanzhe Hu, Yu Wang, and Julian McAuley. 2026b. Evaluating memory in llm agents via incremental multi-turn interactions. In *International Conference on Learning Representations*, volume 2026, pages 156259–156291.
- Cheng Jiayang, Dongyu Ru, Lin Qiu, Yiyang Li, Xuezhi Cao, Yangqiu Song, and Xunliang Cai. 2026. Amemgym: Interactive memory benchmarking for assistants in long-horizon conversations. *arXiv preprint arXiv:2603.01966*.
- Jaehun Jung, Faeze Brahman, and Yejin Choi. 2025. Trust or escalate: Llm judges with provable guarantees for human agreement. In *International Conference on Learning Representations*, volume 2025, pages 3101–3125.
- Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. Memory os of ai agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 25972–25981.
- Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Ivan Rodkin, Dmitry Sorokin, Artyom Sorokin, and Mikhail Burtsev. 2024. Babilong: Testing the limits of llms with long context reasoning-in-a-haystack. *Advances in Neural Information Processing Systems*, 37:106519–106554.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, and 1 others. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474.
- Kai Li, Xuanqing Yu, Ziyi Ni, Yi Zeng, Yao Xu, Zheqing Zhang, Xin Li, Jitao Sang, Xiaogang Duan,

- Xuele Wang, and 1 others. 2026. Timem: Temporal-hierarchical memory consolidation for long-horizon conversational agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 21700–21720.
- Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, and 1 others. 2025. Memos: A memory os for ai system. *arXiv preprint arXiv:2507.03724*.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024a. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, and 1 others. 2024b. Agent-bench: Evaluating llms as agents. In *International Conference on Learning Representations*, volume 2024, pages 52989–53046.
- Yuanjie Lyu, Zhiyu Li, Simin Niu, Feiyu Xiong, Bo Tang, Wenjin Wang, Hao Wu, Huanyong Liu, Tong Xu, and Enhong Chen. 2025. Crud-rag: A comprehensive chinese benchmark for retrieval-augmented generation of large language models. *ACM Transactions on Information Systems*, 43(2):1–32.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870.
- Potsawee Manakul, Adian Liusie, and Mark Gales. 2023. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 9004–9017.
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372.
- Xuanfan Ni, Hengyi Cai, Xiaochi Wei, Shuaiqiang Wang, Dawei Yin, and Piji Li. 2024. Xl²bench: A benchmark for extremely long context understanding with long-range dependencies. *arXiv preprint arXiv:2404.05446*.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. 2023. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*.
- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22.
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*.
- Dehao Tao, Guoliang Ma, Yongfeng Huang, and Minghu Jiang. 2026. Membox: Weaving topic continuity into long-range memory for llm agents. *arXiv preprint arXiv:2601.03785*.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 10014–10037.
- Chonghua Wang, Haodong Duan, Songyang Zhang, Dahua Lin, and Kai Chen. 2024a. Ada-level: Evaluating long-context llms with length-adaptable benchmarks. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3712–3724.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.
- Piaohong Wang, Motong Tian, Jiaxian Li, Yuan Liang, Yuqing Wang, Qianben Chen, Tiannan Wang, Zhicong Lu, Jiawei Ma, Yuchen Eleanor Jiang, and 1 others. 2025. O-mem: Omni memory system for personalized, long horizon, self-evolving agents. *arXiv preprint arXiv:2511.13593*.
- Yidong Wang, Zhuohao Yu, Wenjin Yao, Zhengran Zeng, Linyi Yang, Cunxiang Wang, Hao Chen, Chaoya Jiang, Rui Xie, Jindong Wang, and 1 others. 2024b. Pandalm: An automatic evaluation benchmark for llm instruction tuning optimization. In *International Conference on Learning Representations*, volume 2024, pages 43573–43593.
- Yuyao Wang, Zhongjian Zhang, Mo Chi, Kaichi Yu, Yuhao Li, Miao Peng, Bing Tong, Chen Zhang, Yan Zhou, and Jia Li. 2026. Evomembench: Benchmarking agent memory from a self-evolving perspective. *arXiv preprint arXiv:2605.18421*.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2024. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2026. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems*, 38:17577–17604.

BY Yan, Chaofan Li, Hongjin Qian, Shuqi Lu, and Zheng Liu. 2025. General agentic memory via deep research. *arXiv preprint arXiv:2511.18423*.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, and 1 others. 2025. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*.

Lianghui Zhu, Xinggang Wang, and Xinlong Wang. 2025. Judgelm: Fine-tuned large language models are scalable judges. In *International Conference on Learning Representations*, volume 2025, pages 51257–51296.

A Additional Related Work

From long-context evaluation to memory-system evaluation. Early long-context benchmarks primarily test whether a model can locate and reason over evidence already present in a long input sequence, as in LONGBENCH, ADA-LEVAL, XL²BENCH, and BABILONG (Bai et al., 2024; Wang et al., 2024a; Ni et al., 2024; Kuratov et al., 2024). Long-term memory benchmarks shift the focus from passive reading to persistent interaction: the system must decide what to store, how to update it, and when to retrieve it across sessions. Representative datasets such as LOCOMO and LONGMEMEVAL establish this setting for conversational recall, temporal reasoning, knowledge updates, and abstention (Maharana et al., 2024; Wu et al., 2024). This distinction is important for EvalMem: our target is not long-context saturation by itself, but diagnosing failures in externalized memory loops after information has been written, reorganized, and later queried.

Static, incremental, and on-policy memory benchmarks. Recent benchmarks form a progression from fixed-history evaluation to increasingly interactive settings. LOCOMO and LONGMEMEVAL largely evaluate systems against fixed or pre-recorded histories (Maharana et al., 2024; Wu et al., 2024). MEMORYAGENTBENCH and MEMORYBENCH broaden the task surface toward

incremental multi-turn interaction, continual learning, and user feedback (Hu et al., 2026b; Ai et al., 2025). HALUMEM focuses on hallucination under memory extraction, updating, and question answering (Chen et al., 2025), while AMEMGYM moves evaluation on-policy so that the tested assistant’s own interaction trajectory affects later memory use (Jiayang et al., 2026). Very recent work continues this move toward self-evolving settings (Wang et al., 2026). DynaMem-Bench is closest in spirit to the interactive line, but differs in objective: rather than maximizing environmental realism alone, it introduces a controlled must-expose schedule so different systems remain comparable at the level of fact exposure while still generating their own dialogue trajectories.

From stage-aware evaluation to failure attribution. Several prior works already recognize that end-to-end answer accuracy is too coarse for understanding memory systems. LONGMEMEVAL discusses indexing, retrieval, and reading (Wu et al., 2024); HALUMEM decomposes hallucinations across extraction, update, and question answering (Chen et al., 2025); AMEMGYM reports writing, reading, and utilization metrics (Jiayang et al., 2026); and MEMORYAGENTBENCH evaluates broader memory capabilities through multi-turn interaction (Hu et al., 2026b). These studies motivate operation-aware evaluation, but they do not fully solve attribution for deployed memory systems, where a single failure can span multiple operations at once. A fact may be encoded ambiguously, retrieved too late, and then ignored by the generator; collapsing such a case into one bucket hides the coupled nature of the error. EvalMem therefore frames diagnosis as a parallel, multi-label attribution problem rather than a single-stage classification problem.

Memory architectures and retrieval-side optimization. Recent memory systems differ substantially in how they store, organize, and retrieve past interactions. Hierarchical and temporal systems maintain multi-level summaries or time-aware indices (Kang et al., 2025; Li et al., 2026); operating-system-style designs separate memory types and lifecycle operations (Li et al., 2025; Hu et al., 2026a); and other systems use modular memory blocks, user-centric profiles, or agentic memory construction (Tao et al., 2026; Wang et al., 2025; Yan et al., 2025; Xu et al., 2026). Production-oriented systems similarly combine ex-

PLICIT memory stores with retrieval-augmented generation (Chhikara et al., 2025; Rasmussen et al., 2025). This diversity makes direct architectural comparison difficult, and leaderboard accuracy may obscure which component is responsible for an observed gain or failure. It also leaves retrieval under-examined: many memory systems invest heavily in what to store, but still expose the LLM to a relatively flat top- K retrieved context. Retrieval-augmented generation and agentic retrieval methods provide useful building blocks for high-recall search and iterative query refinement (Lewis et al., 2020; Asai et al., 2024; Trivedi et al., 2023), but they are usually proposed as task-solving mechanisms rather than as diagnostic instruments for memory evaluation. In contrast, EvalMem uses an adapted agentic RAG procedure as a post-hoc probe for store-level diagnosis, and MemWiki as a diagnosis-guided auxiliary structure to test whether retrieval defects can be turned into actionable system improvements.

B Theory and Definitions

B.1 Adapted AgenticRAG Pseudocode

Algorithm 1: adapted AgenticRAG diagnostic routine (sketch)

Input: query Q , key fact F_{key} , store $\mathcal{M}$.
Output: candidate pool, stop reason.
 $\text{pool} \leftarrow \emptyset$; $\text{prev} \leftarrow \emptyset$; $s \leftarrow 0$.
for $t = 1 \dots T_{\text{max}}$ **do**
 $\text{views} \leftarrow \text{PLAN}(Q, F_{\text{key}}, \text{diag})$
 $\text{raw} \leftarrow \text{PARALLEL-RETRIEVE}(\text{views}, \mathcal{M}, \text{top-}K)$
 $\text{fused} \leftarrow \text{RRF-FUSE}(\text{raw})$; $\text{ranked} \leftarrow \text{RERANK}(\text{fused})$
 $\text{pool} \leftarrow \text{MERGE}(\text{pool}, \text{ranked})$
 if $|\text{pool}| \geq 50$ **return** pool , POOL-FULL
 if $\text{CONVERGED}(\text{ranked}, \text{prev})$ **then** $s \leftarrow s + 1$;
 if $s \geq 2$ **return** pool , CONVERGED
 $\text{prev} \leftarrow \text{ranked}$
return pool , MAX-ITER

The adapted AgenticRAG routine is a bounded multi-view recall procedure with deterministic convergence. The diagnosis vector diag summarises whether the previous round surfaced any candidate overlapping with F_{key} and, if not, what sub-aspects of F_{key} remain unanchored. Views span dense, sparse (BM25), and rewritten paraphrases derived from F_{key} ; RRF-FUSE is reciprocal-rank fusion. The two stop conditions (POOL-FULL when the candidate pool exceeds 50 records; CONVERGED when two consecutive rounds produce no new top-ranked candidates) guarantee termination in at most $T_{\text{max}} = 4$ rounds, with empirical mean of 1.74 ($\sigma = 0.81$) across 1540 LoCoMo POS queries. We set $T_{\text{max}} = 4$ because 87.9% of queries termi-

nate via convergence or pool-full before reaching this cap (the remaining 12.1% trigger MAX-ITER); raising the cap to $T_{\text{max}} = 6$ would help only this long tail at near-linear cost while leaving the global MISS rate essentially unchanged.

B.2 MemWiki Pseudocode

Algorithm 2: MemWiki build (offline)

Input: raw memory $\mathcal{M}$ exported by the tested system.
Output: auxiliary record set $\mathcal{A}$.
 $\mathcal{P} \leftarrow \emptyset$; $\mathcal{A} \leftarrow \emptyset$.
for each record $r \in \mathcal{M}$ **do**
 $\text{page} \leftarrow \text{SOURCE-PAGE}(r)$
 $\text{page.tags} \leftarrow \text{NORMALIZE}(\text{ent}, \text{topic}, \text{time})$
 $\text{page.facts} \leftarrow \text{EXTRACT-ATOMIC-FACTS}(r)$
 $\text{page.hq} \leftarrow \text{GEN-HYPO-QUESTIONS}(r)$
 $\mathcal{P} \leftarrow \mathcal{P} \cup \{\text{page}\}$
 $\mathcal{P} \leftarrow \mathcal{P} \cup \text{AGGREGATE-HUBS}(\mathcal{P})$ // entity / topic / event
 $\mathcal{P} \leftarrow \text{APPLY-VERSIONS}(\mathcal{P})$ // append / rewrite
for each page $p \in \mathcal{P}$ **do**
 $\mathcal{A} \leftarrow \mathcal{A} \cup \text{FLATTEN}(p, \{\text{FACT}, \text{TAG}, \text{QUERY}, \text{SUMMARY}\})$
 each $a \in \mathcal{A}$ carries $\text{src}(a) \subseteq \mathcal{M}$
return $\mathcal{A}$

Algorithm 3: MemWiki at query time (online)

Input: query Q , native retriever Retr , budget K , quota α .
Output: final context C_{final} .
 $C_0 \leftarrow \text{Retr}(Q, \mathcal{M} \cup \mathcal{A})$ // joint pool, native retriever
 $H_{\mathcal{A}} \leftarrow C_0 \cap \mathcal{A}$; $E \leftarrow \bigcup_{a \in H_{\mathcal{A}}} \text{src}(a)$
 $C_1 \leftarrow \text{RERANK}(C_0 \cup E)$ // source-expansion before rerank
 $C_{\text{final}} \leftarrow \text{QUOTA-CAP}(C_1, K, |\cdot \cap \mathcal{A}| \leq \alpha K)$
return C_{final}

Algorithm 2 is build-side and runs once per (system, persona). It is governed by three invariants: (i) the only readable input is $\mathcal{M}$ exported through the system’s adapter — the build never sees $\mathcal{A}_{\text{gold}}$, F_{key} , C_{oracle} , or evidence labels; (ii) every auxiliary record a stores a non-empty back-pointer $\text{src}(a) \subseteq \mathcal{M}$, so an auxiliary hit can always be exchanged for the original evidence at query time; (iii) when a later page rewrites an earlier one, APPLY-VERSIONS writes a new version instead of overwriting — a fact superseded across sessions is recorded as a (old-version, new-version) pair, both retaining their src pointers. FLATTEN produces four record classes per page: FACT (subject-predicate-object-time tuples), TAG (normalised entity / topic / time anchors), QUERY (pre-generated plausible questions answered by the evidence), and SUMMARY (hub-page aggregates spanning multiple sessions). Algorithm 3 is the query-time fusion that realises Eq. (8): the native retriever runs once over $\mathcal{M} \cup \mathcal{A}$, every auxiliary hit is expanded back to its source, and the merged pool is reranked. The

quota cap $\alpha=0.30$ prevents auxiliary records from crowding out original evidence; Table 19 reports the sweep that confirms it is a structural constraint rather than a tuning knob.

B.3 Attribution Rules

Tables 4 and 5 give the complete task-aware mapping from Examiner states to final defect sets. For POS generation failures, $\mathcal{D}_{\text{gen}}^+$ is either $\{\text{GF}\}$ or $\{\text{GRF}\}$; for low-quality POS retrieval hits, $\mathcal{D}_{\text{low}}$ is a non-empty subset of $\{\text{LATE}, \text{NOI}\}$. For POS, HIT-LOW means the evidence is retrieved but ranked too late or diluted by noise; HIT-CLEAN means the hit passes both checks. For NEG, MISS/MISS/PASS is the healthy path. The Generation Examiner uses a comparison schema in implementation: it records both the online answer produced under the system’s own retrieved context and the oracle answer produced under C_{oracle} , while the attribution tables use the oracle-side generation state. Full defect-code names are listed in Table 1. The POS/NEG matrices below specify the mapping from probe states to defect codes.

B.4 Defect Codes

The full taxonomy of 11 defect codes was summarised in Table 1. We collect their full natural-language definitions here for reproducibility.

- **EM (Encoding Missing).** The Encoding Examiner reports MISS: no (k, v) in $\mathcal{M}$ matches k_{target} .
- **EA (Encoding Ambiguous).** A matching key exists but v is unresolved (e.g. pronoun, date placeholder).
- **EW (Encoding Wrong).** A matching key exists but $v \neq v_{\text{target}}$.
- **DMP (Dirty Memory Pollution).** A NEG-side hallucinated fact is in $\mathcal{M}$.
- **RF (Retrieval Failure).** $S_{\text{enc}} = \text{EXIST}$ but $F_{\text{key}} \not\subseteq C_{\text{original}}$.
- **LATE (Retrieval Late).** $F_{\text{key}} \subseteq C_{\text{original}}$ but $\text{rank}(F_{\text{key}}) > \tau_{\text{rank}} = 5$.
- **NOI (Retrieval Noise).** $F_{\text{key}} \subseteq C_{\text{original}}$ but $\text{snr}(C_{\text{original}}) < \tau_{\text{snr}} = 0.20$.
- **NIR (Noise-Induced Retrieval).** A NEG query surfaces a relevant-looking but misleading record (e.g. stale evidence after rollback).

- **GH (Generation Hallucination).** A NEG query is answered (model fails to refuse) under oracle context.
- **GF (Generation Faithfulness).** POS query: oracle context contains the answer, but the model uses parametric knowledge instead.
- **GRF (Generation Reasoning Failure).** POS query: model reads the oracle context but reasons incorrectly (multi-hop, temporal, etc.).

C Experimental Results

C.1 Full Cross-Backbone, Cross-Dataset Results

Table 6 reports the complete $7 \times 3 \times 3$ grid: seven memory systems, three datasets (as vertical Sets), and three answer-generation backbones, each with Final Acc and the three layer-aggregate defect rates. The aggregate row at the bottom of each Set is the unweighted mean over the seven systems. Figures 4–5 (heatmaps) resolve every layer total into its 11 defect codes on the two primary datasets; we omit LONGMEMEVAL-S from the sub-code resolution because its NEG subsample ($N_{\text{NEG}} = 30$) is too small to support reliable per-code analysis (the layer totals remain in Table 6).

C.2 Native-Embedding Robustness

The main comparison standardizes the embedding encoder to isolate operation-level differences under a common retrieval representation. To assess sensitivity to released defaults, we rerun the Set 1 protocol with each system’s native encoder and use the same encoder in the Encoding-Examiner diagnostic retrieval. As shown in Table 7, systems with a changed encoder shift in accuracy by 1.2–6.8 pp, while Retrieval remains the most frequently attributed layer; TiMem uses the standardized encoder natively. The rerun changes only the embedding encoder and is not a complete study of every released native configuration.

Backbone sensitivity is bounded and structurally consistent. Replacing GPT-4O-MINI with GPT-4.1 reduces mean Final Acc gap by +2.8/+4.7/+3.7 pp on the three datasets, with the gain absorbed across all three layers (Enc $-1.5/-1.4/-1.4$ pp; Ret $-1.9/-3.1/-1.0$ pp; Gen $-1.9/-1.2/-1.3$ pp). QWEN3-30B sits within ± 1.3 pp of the default on Acc and within ± 1 pp on every layer. The descriptive aggregate pattern is preserved across all three backbones: Retrieval

ID	S_{enc}	S_{ret}	S_{gen}	$\mathcal{D}_{\text{total}}$	Attribution meaning
P1	MISS	MISS	PASS	{EM}	Fact absent from store; retrieval miss is shielded.
P2	MISS	MISS	FAIL	{EM} $\cup$ $\mathcal{D}_{\text{gen}}^+$	Storage failure plus oracle-side generation failure.
P3	CORRUPT AMBIG	Any	PASS	{EA}	Stored evidence is referentially ambiguous.
P4	CORRUPT AMBIG	Any	FAIL	{EA} $\cup$ $\mathcal{D}_{\text{gen}}^+$	Ambiguous evidence plus generation failure.
P5	CORRUPT WRONG	Any	PASS	{EW}	Stored value is wrong.
P6	CORRUPT WRONG	Any	FAIL	{EW} $\cup$ $\mathcal{D}_{\text{gen}}^+$	Wrong value plus generation failure.
P7	EXIST	MISS	PASS	{RF}	Fact is stored but native retrieval misses it.
P8	EXIST	MISS	FAIL	{RF} $\cup$ $\mathcal{D}_{\text{gen}}^+$	Retrieval failure plus generation failure.
P9	EXIST	HIT-LOW	PASS	$\mathcal{D}_{\text{low}}$	Evidence appears, but ranking or SNR is poor.
P10	EXIST	HIT-LOW	FAIL	$\mathcal{D}_{\text{low}} \cup \mathcal{D}_{\text{gen}}^+$	Low-quality retrieval plus generation failure.
P11	EXIST	HIT-CLEAN	PASS	$\emptyset$	Healthy POS path.
P12	EXIST	HIT-CLEAN	FAIL	$\mathcal{D}_{\text{gen}}^+$	Memory path is healthy; failure is generation-side.

Table 4: Attribution matrix for POS questions. Retrieval defects are counted only after the source fact is confirmed to exist in memory; otherwise retrieval miss is a consequence of encoding failure.

ID	S_{enc}	S_{ret}	S_{gen}	$\mathcal{D}_{\text{total}}$	Attribution meaning
N1	DIRTY	NOISE	PASS	{DMP}	Pseudo-fact is already in memory; retrieval noise is attributed to storage pollution.
N2	DIRTY	NOISE	FAIL	{DMP, GH}	Dirty memory plus unsupported generation.
N3	DIRTY	MISS	PASS	{DMP}	Store is polluted, although native retrieval does not surface it.
N4	DIRTY	MISS	FAIL	{DMP, GH}	Dirty memory plus unsupported generation without retrieved support.
N5	MISS	NOISE	PASS	{NIR}	Clean store, but retrieval surfaces a misleading record.
N6	MISS	NOISE	FAIL	{NIR, GH}	Misleading retrieval plus unsupported generation.
N7	MISS	MISS	PASS	$\emptyset$	Healthy NEG path: clean memory, clean retrieval, refusal.
N8	MISS	MISS	FAIL	{GH}	Model fabricates without memory or retrieval support.

Table 5: Attribution matrix for NEG questions. DMP is assigned when an unsupported pseudo-fact is already written into memory; NIR is assigned only when a clean store is followed by misleading retrieval.

has the highest attributed rate in every evaluated (dataset, backbone) cell, under the full-query denominator and the error gate in §2.2.

Ranking stability. Spearman rank correlation between any two backbones on a single dataset ranges from 0.929 to 0.964. Two near-tied flips appear: TiMem $\leftrightarrow$ O-Mem under GPT-4.1 on LOCoMo and DynaMem-Bench (both pairs sepa-

rated by ≤ 0.6 pp); MemoryOS $\leftrightarrow$ MemBox under QWEN3-30B on LOCoMo (separated by 0.6 pp). The replication of the TiMem $\leftrightarrow$ O-Mem flip on the dynamic dataset suggests it is driven by GPT-4.1’s ability to exploit O-Mem’s entity-graph index.

Stronger backbones disproportionately help generation. On LOCoMo, the largest relative shrinkage from GPT-4O-MINI to GPT-4.1 is in

System	GPT-4O-MINI (default)				GPT-4.1				QWEN3-30B-INSTRUCT			
	Acc $\uparrow$	Enc $\downarrow$	Ret $\downarrow$	Gen $\downarrow$	Acc $\uparrow$	Enc $\downarrow$	Ret $\downarrow$	Gen $\downarrow$	Acc $\uparrow$	Enc $\downarrow$	Ret $\downarrow$	Gen $\downarrow$
Set 1: LoCoMo $N = 1986$												
TiMem	75.3	7.1	17.0	6.0	<u>77.8</u>	5.5	15.4	4.1	74.2	7.8	17.6	6.5
O-Mem	<u>74.9</u>	4.9	<u>19.6</u>	<u>6.1</u>	78.1	3.7	<u>17.5</u>	<u>4.2</u>	<u>73.8</u>	5.5	<u>20.2</u>	<u>6.7</u>
EverMemOS	72.4	8.6	20.0	6.3	75.2	6.6	17.8	4.4	71.0	9.3	20.7	6.9
MemOS	70.1	8.1	21.0	6.5	73.1	6.1	18.7	4.7	69.2	8.7	21.8	7.1
MemBox	63.2	<u>5.9</u>	25.7	6.9	65.7	<u>5.4</u>	24.0	5.2	61.7	<u>6.5</u>	26.6	7.5
MemoryOS	60.8	9.3	26.0	6.6	63.8	7.5	24.4	4.8	62.3	9.9	25.3	7.2
GAM	60.2	10.1	25.7	7.1	62.8	8.6	23.8	5.2	59.7	10.8	26.6	7.8
mean	68.1	7.7	22.1	6.5	70.9	6.2	20.2	4.6	67.4	8.4	22.7	7.1
Set 2: LONGMEMEVAL-S $N = 500$ (layer totals only; NEG subsample $N_{\text{NEG}} = 30$ too small for 11-code analysis)												
EverMemOS	82.6	5.6	12.0	4.4	86.4	4.4	9.4	3.4	81.0	6.0	13.0	4.8
O-Mem	<u>81.4</u>	<u>6.0</u>	<u>12.8</u>	<u>4.8</u>	<u>85.6</u>	<u>4.6</u>	<u>10.0</u>	<u>3.6</u>	<u>80.2</u>	<u>6.4</u>	<u>13.6</u>	<u>5.0</u>
MemBox	77.0	7.4	15.8	5.2	81.4	6.0	13.0	4.0	75.2	8.0	16.8	5.8
GAM	76.6	7.6	16.0	5.4	81.0	6.2	13.2	4.2	76.0	7.8	16.8	5.6
TiMem	76.0	7.6	16.6	5.6	80.0	6.4	13.8	4.4	74.8	8.2	17.4	6.0
MemOS	65.0	11.2	24.0	6.0	73.2	8.6	18.4	4.8	64.2	11.2	24.2	6.4
MemoryOS	55.4	14.4	30.6	6.2	58.8	13.2	28.4	5.0	56.8	14.0	29.8	6.6
mean	73.4	8.5	18.3	5.4	78.1	7.1	15.2	4.2	72.6	8.8	18.8	5.7
Set 3: DynaMem-Bench $N = 962$												
TiMem	71.2	<u>7.6</u>	19.0	4.3	<u>74.5</u>	<u>6.0</u>	18.0	3.2	69.6	<u>8.1</u>	19.9	4.9
O-Mem	<u>68.9</u>	7.5	<u>21.5</u>	<u>4.5</u>	75.1	5.5	<u>18.5</u>	3.0	<u>67.7</u>	<u>8.1</u>	<u>22.3</u>	<u>5.1</u>
MemOS	62.5	9.0	25.4	4.9	65.7	8.0	24.9	3.6	60.3	9.9	26.4	5.5
EverMemOS	61.0	9.6	26.1	4.7	64.3	8.5	25.5	3.4	59.0	10.3	27.1	5.3
MemoryOS	56.4	11.0	28.4	5.1	59.8	9.5	28.0	3.8	57.5	10.9	27.9	5.7
MemBox	55.9	10.0	29.1	5.5	58.9	9.0	28.5	4.3	54.2	10.7	30.1	6.1
GAM	49.6	13.0	32.0	5.9	53.0	11.5	31.0	4.7	48.1	13.7	33.1	6.5
mean	60.8	9.7	25.9	5.0	64.5	8.3	24.9	3.7	59.5	10.2	26.7	5.6

Table 6: Full $7 \times 3 \times 3$ matrix of end-to-end accuracy (Acc) and the three layer-aggregate defect rates (Enc, Ret, Gen) across seven memory systems, three datasets (Sets 1–3), and three answer-generation backbones. Defect codes are assigned only to incorrectly answered outputs; all defect rates are sample-level union incidences with the full-query denominator; defect codes may overlap. **Bold red** marks the best per column within each Set (highest Acc, lowest defect); blue underline marks the second-best; ties receive the same marker at every tied position. For LONGMEMEVAL-S (Set 2) we report only Acc and the three layer totals: with $N_{\text{NEG}} = 30$, the smallest 11-code cell would be unstable beyond statistical reliability (Wilson 95% CI ≥ 25 pp); the layer-total view remains diagnostic and is the primary use of this dataset as a generalisation check on LOCoMo. Systems within each Set are ordered by Acc under the default backbone (GPT-4O-MINI). The companion 11-code resolution for LOCoMo and DynaMem-Bench appears in Figs. 4–5.

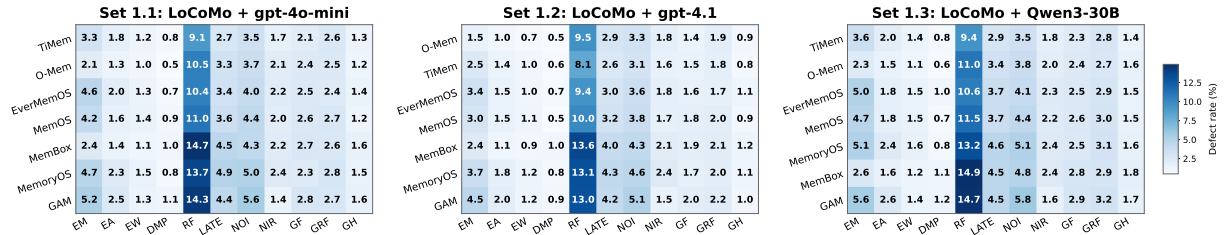


Figure 4: Heatmap view of the full 11-code sub-code breakdown on LOCoMo across three answer-generation backbones (Set 1.1–1.3). Rows are memory systems, columns are defect codes, and each cell shows the sample-level union defect rate (%) over the full $N = 1986$ POS+NEG query pool; codes are assigned only to incorrectly answered outputs. Color indicates defect magnitude (lighter is lower, darker is higher), while text annotations preserve exact values. Green boxes mark per-column best (lowest defect) within each Set; blue boxes mark second-best (ties are all marked).

the Gen layer (-29% , from 6.5 to 4.6), then Enc (-19% , from 7.7 to 6.2), then Ret (-9% , from 22.1 to 20.2). This is consistent with generation being the layer most directly exercised by backbone capability, whereas encoding and retrieval are largely the memory system’s own responsibility.

C.3 Threshold Calibration and Sensitivity

The thresholds τ_{rank} , τ_{snr} , and τ_H affect only the Retrieval subcodes LATE, NOI, and NIR; they do not change Encoding, Generation, or RF assignments. We selected them with system-stratified

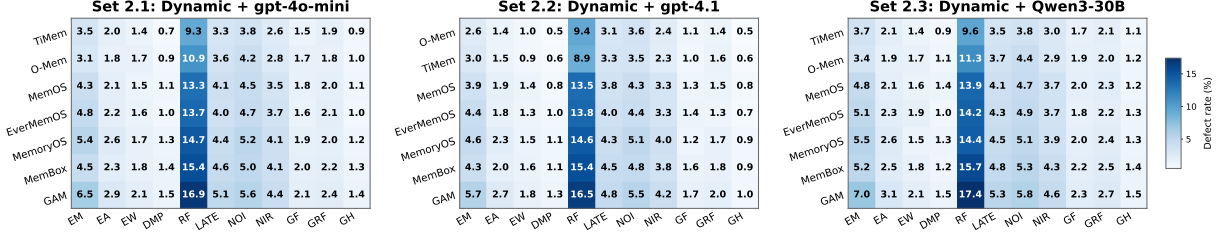


Figure 5: Heatmap view of the full 11-code sub-code breakdown on DynaMem-Bench across three answer-generation backbones (Set 2.1–2.3). Values are sample-level union defect rates (%) over the full $N = 962$ POS+NEG query pool; defect codes are assigned only to incorrectly answered outputs. The NIR column remains visibly darker than in LOCOMO, reflecting the larger share of retrieval defects caused by stale-evidence inference risk. Green and blue boxes denote per-column best and second-best systems within each Set.

System	Released/native embedding	Acc S/N	Δ Acc	Enc/Ret/Gen (S)	Enc/Ret/Gen (N)
TiMem	Qwen3-Embedding-0.6B (same as standardized)	75.3/75.3	0.0	7.1/17.0/6.0	7.1/17.0/6.0
O-Mem	all-MiniLM-L6-v2	74.9/69.8	-5.1	4.9/19.6/6.1	6.2/23.4/6.7
EverMemOS	Qwen3-Embedding-4B	72.4/76.0	+3.6	8.6/20.0/6.3	7.5/17.5/5.9
MemOS	nomic-embed-text:latest	70.1/72.9	+2.8	8.1/21.0/6.5	7.2/18.8/6.1
MemBox	text-embedding-3-small	63.2/56.4	-6.8	5.9/25.7/6.9	7.5/30.8/7.6
MemoryOS	all-MiniLM-L6-v2	60.8/63.2	+2.4	9.3/26.0/6.6	8.5/23.7/6.3
GAM	BGE-M3	60.2/61.4	+1.2	10.1/25.7/7.1	9.7/24.6/6.9

Table 7: Native-embedding robustness on LOCOMO with GPT-4O-MINI. S/N denotes the standardized/native embedding runs; Δ is native minus standardized accuracy in percentage points. Only the embedding encoder used by the system’s embedding-based retrieval/index and the corresponding Encoding-Examiner diagnostic retrieval is changed; its vectors/index entries are rebuilt as needed, while the answer backbone, Top- K , EvalMem judges, memory construction, index type/structure, retrieval logic, and all other settings remain fixed. TiMem uses the standardized encoder natively, and Retrieval remains the most frequently attributed layer in every row. Layer rates are unions assigned only to incorrectly answered outputs, with the full-query denominator. This is an embedding-only ablation, not a complete per-system native-configuration study.

human calibration on fixed pilot samples before aggregating test results. Table 8 reports one-at-a-time sweeps and calibration macro-F1; the selected values are 5, 0.20, and 0.55, respectively.

C.4 Five-run Diagnostic Stability

To isolate stochasticity in the evaluator, we hold the baseline systems’ LOCOMO outputs fixed and rerun the Encoding, Retrieval, and Generation Examiners together with the Attribution Agent five times under the Set 1 configuration. Table 9 reports the mean and standard deviation of each layer rate in percentage points and Fleiss’ κ across the five run-level diagnoses. For auditability, Tables 10–12 resolve the same layer summaries into their constituent defect codes. All entries are mean $\pm$ SD in percentage points over the five evaluator reruns; the final column in each table is the corresponding layer total.

C.5 Diagnostic Token, Call, and Latency Overhead

We measure per-example EvalMem diagnostic overhead under the Set 1 protocol; K denotes 10^3 tokens and p50/p95 are per-example critical-path latencies. Table 13 gives per-system totals, while Table 14 decomposes the mean into Agentic-RAG (AR), the Encoding judge (EJ), Retrieval, Generation, and Attribution. AR and EJ together form the Encoding path. The three Examiners execute in parallel and Attribution follows them; hence the critical path is approximately $\max(t_E, t_R, t_G) + t_A$, where AR is an internal component of t_E , rather than the sum of all listed latencies.

Mechanism-level signals from the sub-code view. The sub-code resolution makes several mechanism-level shifts visible. On LOCOMO the TiMem $\leftrightarrow$ O-Mem flip under GPT-4.1 is driven by O-Mem’s lower EM/EA/EW (1.5/1.0/0.7 vs. TiMem’s 2.5/1.4/1.0), not by retrieval differences

Threshold	Candidates	Controlled sub-code rate (%)	Calibration macro-F1	Selected
τ_{rank}	3/5/7/9	6.3/3.8/2.6/1.7	0.78/0.84/0.76/0.58	5
τ_{snr}	0.10/0.15/0.20/0.25/0.30	1.8/2.9/4.4/6.0/8.1	0.61/0.75/0.82/0.79/0.74	0.20
τ_H	0.45/0.50/0.55/0.60/0.65	3.2/2.5/2.0/1.6/1.3	0.79/0.82/0.84/0.78/0.69	0.55

Table 8: Threshold sensitivity and calibration under the Set 1 protocol. One threshold is varied at a time; the controlled sub-code rate is the unweighted mean over seven systems. LATE/NOI calibration uses 200 erroneous POS–HIT pairs and NIR calibration uses 200 NEG pairs. Two annotators label independently and a third adjudicates disagreements; the selected value maximizes calibration macro-F1.

System	Enc mean $\pm$ SD	Ret mean $\pm$ SD	Gen mean $\pm$ SD	Fleiss’ κ
TiMem	7.16 $\pm$ 0.17	16.84 $\pm$ 0.22	6.09 $\pm$ 0.11	0.93
O-Mem	4.97 $\pm$ 0.14	19.72 $\pm$ 0.20	6.05 $\pm$ 0.10	0.91
EverMemOS	8.53 $\pm$ 0.19	20.17 $\pm$ 0.23	6.39 $\pm$ 0.11	0.93
MemOS	8.18 $\pm$ 0.18	20.91 $\pm$ 0.21	6.57 $\pm$ 0.13	0.94
MemBox	6.03 $\pm$ 0.16	25.86 $\pm$ 0.25	6.83 $\pm$ 0.12	0.92
MemoryOS	9.37 $\pm$ 0.21	26.12 $\pm$ 0.24	6.77 $\pm$ 0.14	0.89
GAM	10.05 $\pm$ 0.20	25.82 $\pm$ 0.25	7.16 $\pm$ 0.13	0.90

Table 9: Five-run diagnostic stability under the Set 1 configuration (LOCOMO with GPT-4O-MINI). Baseline systems’ outputs are held fixed while the full EvalMem diagnostic pipeline is rerun five times. Layer values are mean $\pm$ SD in percentage points, using the error gate and full-query denominator in §2.2; Fleiss’ κ measures agreement across the five run-level diagnoses. This is evaluator-run stochasticity, not memory-system training or evaluation-seed variance.

(TiMem’s RF 8.1 is in fact lower than O-Mem’s 9.5); the rank-1 system flips because the entity-graph encoding savings outweigh the retrieval gap once the backbone is strong enough to exploit them. On DynaMem-Bench the NIR code carries a disproportionate share of retrieval defects across every backbone (2.3–4.6%, compared with $\leq 2.4\%$ on LOCOMO), consistent with rollback / cascade events producing stale but relevant-looking evidence.

C.6 Adapted AgenticRAG Mechanism Detail

The controlled study is run on an *oracle vector store*: the full LOCOMO10 dialogue is chunked at a fixed turn-level granularity and indexed with Qwen3-Embedding-0.6B, decoupling the diagnostic routine from any tested system’s ingestion pipeline so that retrieval-side effects can be isolated from storage-side ones.

Table 15 reports the full C_1 – C_4 controlled study together with the leave-one-out breakdown of C_4 . The four conditions disentangle two orthogonal ingredients: (i) the adapted agentic-RAG pipeline mechanisms—multi-view dense retrieval, BM25 sparse view, LLM rerank, and multi-round refinement (C_2, C_4) vs. a vanilla single-view dense retriever (C_1, C_3); (ii) the evidence-aware query that injects the oracle key fact F_{key} into the query (C_3, C_4) vs. the raw user question Q alone

(C_1, C_2). The two ingredients are nearly independent: the pipeline mechanisms contribute -11.4 pp (between C_1 and C_2) and -6.9 pp (between C_3 and C_4). McNemar tests confirm both improvements are highly significant.

Leave-one-out near-additivity. The leave-one-out section removes each ingredient from C_4 in isolation. The evidence-aware query is the single largest contributor at $+14.0$ pp when removed (which reverts C_4 exactly back to the C_2 MISS rate). The four adapted agentic-RAG pipeline mechanisms—multi-view dense retrieval, the LLM reranker, the multi-round loop, and the BM25 sparse view—contribute $+2.5$, $+2.0$, $+1.4$, and $+0.7$ pp respectively. Their sum is 6.6 pp; the net contribution of the four mechanisms over C_3 is 6.9 pp. The 0.3 pp gap indicates the four pipeline mechanisms are near-additive, each covering an orthogonal failure mode.

Convergence statistics. Across the 1540 POS queries with $T_{\text{max}} = 4$, the mean number of rounds taken before termination is 1.74 ($\sigma = 0.81$). 87.9% of queries terminate via convergence (62.9% by the new-ratio criterion $\epsilon < 0.10$, 25.0% by the Jaccard criterion ≥ 0.85 for two consecutive rounds); the remaining 12.1% hit T_{max} and are concentrated on multi-hop temporal questions where the key fact spans multiple sessions. The pool-full stop does

System	EM	EA	EW	DMP	Enc total
TiMem	3.45±0.21	1.67±0.22	1.31±0.20	0.73±0.15	7.16±0.17
O-Mem	2.11±0.05	1.38±0.19	1.01±0.04	0.47±0.05	4.97±0.14
EverMemOS	4.51±0.16	1.96±0.09	1.34±0.07	0.72±0.04	8.53±0.19
MemOS	4.14±0.14	1.69±0.23	1.54±0.18	0.81±0.16	8.18±0.18
MemBox	2.31±0.10	1.43±0.08	1.21±0.20	1.09±0.19	6.03±0.16
MemoryOS	4.60±0.15	2.33±0.10	1.65±0.16	0.79±0.05	9.37±0.21
GAM	5.18±0.11	2.37±0.23	1.42±0.15	1.09±0.08	10.05±0.20

Table 10: Five-run diagnostic stability for the Encoding layer under Set 1. Each cell is a mean±SD in percentage points over the same five evaluator reruns; the four code columns are the resolution of the Enc total. The error gate and full-query denominator are those in §2.2.

System	RF	LATE	NOI	NIR	Ret total
TiMem	9.30±0.22	2.71±0.08	3.30±0.29	1.53±0.18	16.84±0.22
O-Mem	10.56±0.18	3.11±0.27	3.90±0.27	2.15±0.15	19.72±0.20
EverMemOS	10.43±0.13	3.21±0.31	4.13±0.31	2.40±0.27	20.17±0.23
MemOS	11.20±0.23	3.58±0.07	4.21±0.28	1.92±0.24	20.91±0.21
MemBox	14.89±0.32	4.49±0.09	4.33±0.09	2.15±0.10	25.86±0.25
MemoryOS	13.87±0.27	4.70±0.32	4.98±0.08	2.57±0.29	26.12±0.24
GAM	14.46±0.32	4.59±0.34	5.46±0.17	1.31±0.20	25.82±0.25

Table 11: Five-run diagnostic stability for the Retrieval layer under Set 1. Each cell is a mean±SD in percentage points over the same five evaluator reruns; the four code columns are the resolution of the Ret total. The error gate and full-query denominator are those in §2.2.

not fire on this set.

C.7 MemWiki Mechanism Detail

Table 16 reports the four AuxRecord classes turned on one at a time, alongside their sum, the all-four configuration (used in the main paper), and the resulting *crowd loss*: the share of singleton gain that is dropped when all four classes compete for the same 30% quota. Three observations follow.

Factual records dominate. For every system, the factual class is the largest singleton contributor (between +0.4 and +2.7 pp). Tag and aggregate-summary classes contribute less because they encode coarser information that often already appears (in some form) inside the system’s native records.

Crowding is real but modest. The crowd loss is −28.6% to −38.8% on the six bottom-row systems — meaningful, but the all-four configuration still recovers the bulk of singleton gain. Only TiMem suffers a much larger −62.5% crowd loss: its native top-level layer (*L5* persona profile, the highest tier of TiMem’s L1–L5 temporal hierarchy that aggregates session content into long-horizon summaries) is already similar to the aggregate-summary class, so the AuxRecord quota largely duplicates information rather than expanding it.

Where MemWiki helps most. The systems with the largest empirical MemWiki gains (MemBox + 4.2, GAM + 4.6) are also those with the highest observed native retrieval-defect rates (RF + LATE) in Table 2. This association is consistent with the auxiliary index being useful when a system’s original store lacks query-shaped or factual decomposition.

Per-system significance of MemWiki gains. Table 17 reports per-system McNemar tests between baseline and MemWiki on LOCoMo. For each system, the 1986 paired observations are partitioned by the four McNemar cells, of which only the discordant pair counts b (MemWiki spoils a previously correct answer) and c (MemWiki recovers a previously wrong answer) contribute to the test statistic $\chi^2 = (b - c)^2 / (b + c)$. The null hypothesis is $b = c$, i.e. MemWiki neither helps nor hurts on aggregate. Six of seven systems show statistically significant improvement ($p < 0.05$, with five at $p < 10^{-3}$); TiMem is the only exception ($\chi^2 = 0.82$, $p = 0.37$), reflecting that its native top-level (*L5*) persona-profile layer already covers most of the indexing surface that MemWiki would otherwise add (the same mechanism documented as the −62.5% crowd loss in Table 16). The Pearson correlation between each system’s retrieval defect rate on LOCoMo and its $\log \chi^2$ is 0.92: MemWiki therefore provides gains that scale with each sys-

System	GF	GRF	GH	Gen total
TiMem	2.13±0.08	2.70±0.14	1.26±0.12	6.09±0.11
O-Mem	2.31±0.10	2.59±0.10	1.16±0.11	6.05±0.10
EverMemOS	2.56±0.11	2.43±0.09	1.41±0.05	6.39±0.11
MemOS	2.51±0.11	2.74±0.12	1.32±0.17	6.57±0.13
MemBox	2.70±0.06	2.65±0.12	1.48±0.16	6.83±0.12
MemoryOS	2.22±0.19	2.90±0.13	1.65±0.19	6.77±0.14
GAM	2.67±0.17	2.75±0.09	1.74±0.17	7.16±0.13

Table 12: Five-run diagnostic stability for the Generation layer under Set 1. Each cell is a mean±SD in percentage points over the same five evaluator reruns; the three code columns are the resolution of the Gen total. The error gate and full-query denominator are those in §2.2.

System	Calls	Input (K)	Output (K)	Critical latency (s)	
				p50	p95
TiMem	8.24	38.0	1.81	11.4	25.4
O-Mem	8.38	40.5	1.88	11.9	26.4
EverMemOS	8.42	44.0	1.97	12.4	27.7
MemOS	8.36	43.0	1.95	12.2	27.3
MemBox	8.64	53.0	2.18	13.8	30.8
MemoryOS	8.52	50.0	2.12	13.4	29.9
GAM	8.80	62.0	2.36	15.6	34.6
<i>Mean</i>	8.48	47.2	2.04	13.0	28.9

Table 13: Per-example EvalMem diagnostic overhead by memory system under the Set 1 protocol. Input/output values are in $K = 10^3$ tokens; calls and token counts are means, and critical latency is the per-example p50/p95 on the parallel-Examiner path followed by Attribution. The mean row is the unweighted mean over seven systems.

tem’s baseline retrieval gap, rather than gains that are uniform across the cohort. The corresponding aggregate χ^2 (145.8) is reported in the caption of Table 18.

Per-system MemWiki intervention. Table 18 reports the per-system end-to-end accuracy and retrieval defect rate before and after MemWiki on LOCOMO with GPT-4O-MINI. Gains are heterogeneous (GAM +4.6 / MemBox +4.2 / TiMem +0.3 pp) and track each baseline’s retrieval gap; the corresponding Δ on the retrieval-defect rate is consistently negative for every system. The summary numbers (+2.5 pp mean Acc, −3.2 pp mean Ret defect) appear in the body of §3.3; this table provides the per-system breakdown.

α -quota robustness. Table 19 sweeps the auxiliary-record quota $\alpha \in \{0.10, \dots, 1.00\}$ on LOCOMO with GPT-4O-MINI. Per-system optima fall in $\{0.20, 0.30, 0.40\}$, but the variation in Final Acc across this band is at most 0.2 pp for every system: the default $\alpha = 0.30$ is therefore within 0.2 pp of every per-system peak, and the plateau $\alpha \in [0.30, 0.40]$ gives identical mean Final Acc (70.6%). The $\alpha = 1.0$ row (no quota) drops mean Final Acc to 64.1% — 4.0 pp below the no-

MemWiki baseline. Without the quota, the rerank promotes auxiliary records past the original evidence, so the retrieval surface becomes net worse. This rules out reading the quota as a tuned hyperparameter: it is a structural constraint that protects the original evidence channel.

Cross-dataset replication on LONGMEMEVAL-S. The same MemWiki configuration repeated on LONGMEMEVAL-S (Table 3 in the main text) yields +2.3 pp mean Acc and −3.2 pp retrieval defect, mirroring the LOCOMO result within 0.2 pp. Aggregate McNemar $\chi^2 = 38.10$ ($p < 10^{-3}$), with five of seven systems significant at $p < 0.05$; the observed association between baseline retrieval defects and accuracy gains is again positive (Pearson $r(\Delta\text{Ret}, \Delta\text{Acc}) = 0.81$). TiMem is again non-significant in the same direction, consistent with its native L5 redundancy.

C.8 Top-K Sensitivity Full Data

Tables 20–21 present the full top- K sweep used in Figure 3(a,b). Three patterns recur across the seven systems.

Hit@ K saturates at $K \approx 10$. The marginal Hit@ K gain between consecutive K values decays from +8.4 pp (at $K = 1 \rightarrow 2$) to +2.3 pp

Component	Calls	Input (K)	Output (K)	Latency (s)	
				p50	p95
Agentic-RAG (AR)	3.48	21.8	0.94	8.2	18.7
Encoding judge (EJ)	1.00	11.6	0.23	3.2	6.8
Retrieval	1.00	8.4	0.31	3.1	6.9
Generation	2.00	3.9	0.39	4.0	8.5
Attribution	1.00	1.44	0.16	1.5	3.4

Table 14: Mean component-level EvalMem diagnostic overhead over seven systems under Set 1. Input/output values are in $K = 10^3$ tokens. Rounded component means sum to the per-system mean up to rounding. Agentic-RAG (AR) and the Encoding judge (EJ) form the Encoding path; the three Examiners are parallel, so component p50/p95 values are not additive for the critical path.

Configuration	MISS/1540	Rate
<i>Controlled study (oracle vector store)</i>		
C_1 Vanilla RAG + Q	459	29.8%
C_2 AgenticRAG + Q	283	18.4%
C_3 Vanilla RAG + $Q + F_{\text{key}}$	174	11.3%
C_4 AgenticRAG + $Q + F_{\text{key}}$ (*)	67	4.4%
<i>Leave-one-out from C_4</i>		
– evidence-aware query ($\rightarrow C_2$)	283	18.4%
– multi-view (dense $\rightarrow 1$)	107	6.9%
– LLM rerank	99	6.4%
– multi-round ($T_{\text{max}} \rightarrow 1$)	89	5.8%
– BM25 sparse view	78	5.1%

Table 15: Adapted AgenticRAG ablation on the LO-COMO POS subset ($N = 1540$). (*) C_4 is the default in all main experiments. The adapted agentic-RAG routine comprises four pipeline mechanisms (multi-view dense retrieval, BM25 sparse view, LLM rerank, multi-round); evidence-aware query is an orthogonal input-level mechanism. Leave-one-out contributions: evidence-aware +14.0, multi-view +2.5, rerank +2.0, multi-round +1.4, BM25 +0.7 pp. McNemar C_1 vs. C_4 $\chi^2 = 298.4$ ($p < 10^{-3}$); C_3 vs. C_4 $\chi^2 = 41.2$ ($p < 10^{-3}$). The four-mechanism leave-one-out sum (6.6 pp) is close to the net pipeline contribution (C_4 vs. C_3 , 6.9 pp), indicating near-additivity with 0.3 pp residual overlap.

($K = 12 \rightarrow 15$), with the elbow between $K = 8$ and $K = 12$ for every system.

Final Acc peaks at $K \in \{8, 10\}$. Beyond the peak, Final Acc drops by an average -2.2 pp from $K = 10$ to $K = 20$ even though Hit@ K continues to rise (Table 20). The discrepancy is explained by Table 21: the NOI rate climbs from 5.5% at $K = 10$ to 12.5% at $K = 20$, so additional candidates are increasingly distractors rather than informative evidence.

The U-shaped retrieval defect rate. The total retrieval-layer defect rate (RF + LATE + NOI) is U-shaped in K , with a flat minimum at $K \in \{8, 10\}$.

The two arms of the U are mechanistically different: at small K retrieval defects come from RF (the key fact is never in the top- K); at large K they come from NOI (the key fact is there but buried in noise). Choosing $K = 10$ as the default lies on the minimum of this curve.

C.9 DynaMem-Bench Construction Pipeline

DynaMem-Bench is produced in two phases: a system-agnostic *offline* phase that fixes everything the fairness comparison requires (personas, state trajectories, must-expose sets, seed utterances, evaluation questions, oracle contexts), and a per-system *on-policy* phase that lets each tested system generate its own dialogues while the offline schedule constrains the facts each system observes. The offline phase is shared across all seven tested systems; the on-policy phase is repeated once per (system, persona).

Persona pool (offline stage 0). We hand-design a grid of 10 personas spanning six structured attributes: age band (22–67), education level (high-school to doctoral), career stage (student / early-career / mid-career / freelance / retired / caregiver), communication style (colloquial, concise, narrative, technical, indirect), state-change propensity $\rho \in \{0.05, 0.15, 0.30\}$ that governs single-point updates, and NEG sensitivity (revises past statements, privacy-averse, vague). GPT-4O renders each persona into a ~ 200 -token backstory used as the semantic anchor for downstream stages.

Question sampling (stage 1). For each persona, GPT-4O samples 100 candidate questions weighted by the persona’s preferred domains. Each candidate carries a structured `required_info` field listing the one or two state variables the gold answer depends on; these dependencies are the seed for the state schema in stage 2.

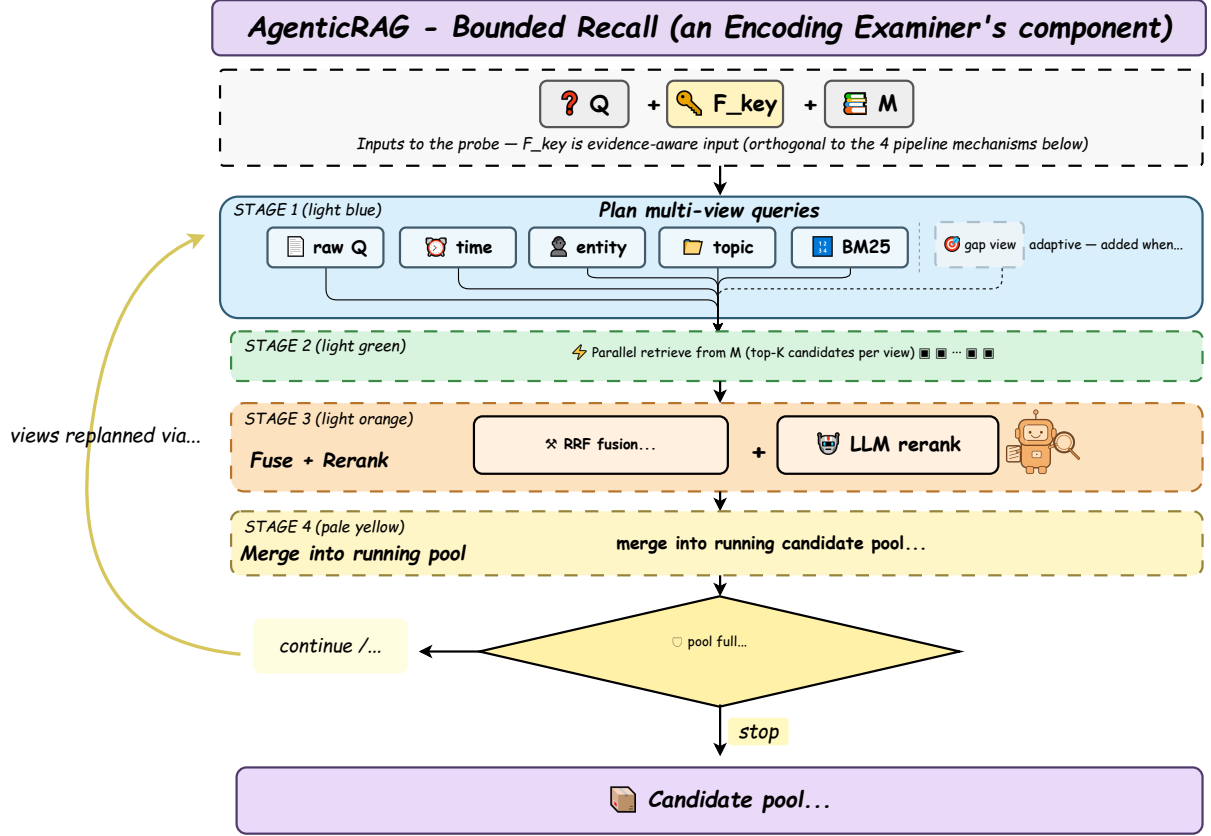


Figure 6: Adapted AgenticRAG diagnostic pipeline. For each (Q, F_{key}) , the agent plans a set of views (dense, BM25, time, entity, topic, plus an on-demand gap view), routes a granularity-aware top- K per view, fuses with RRF, and reranks by evidence to F_{key} . The loop terminates by pool-saturation, Jaccard stability, or T_{max} , so it bounds observation MISS without false confidence on truly absent facts.

State schema with double cores and cascade (stage 2). GPT-4O consolidates the union of required_info fields into a state schema Σ with 5–8 variables per persona, each with an enumerated value space. The schema designates a *main-core* variable (e.g. city) and one or more *auxiliary-core* variables (e.g. commute_mode) annotated with a cascade_lag (immediate vs. delayed) and a written cascade reason. Auxiliary cores are the seed for the cascade subset in DynaMem-Bench.

State evolution trajectory (stage 3). A deterministic script advances $\sigma_0 \rightarrow \sigma_{10}$ according to Table 22: sessions 1–5 keep σ_0 for stable recall; session 6 flips the main-core (forcing every state-tracking question to update); session 7 executes the auxiliary-core cascade with the schema’s stated lag; sessions 8 and 10 apply ρ -driven single-point updates on non-core variables; for 30% of personas (selected by persona_id hash), session 9 rolls the main-core back to its σ_0 value. The script also emits the per-session interference set I_t — the stale prior values that the system must now treat as superseded — which serves as the source of trap answers

for NEG queries probing stale evidence after a flip.

Must-expose set and seed utterance (stage 4).

For each session t , the must-expose set E_t enumerates every (variable, value) pair that first appears or changes in σ_t , annotated with a per-pair tone_hint matching the persona’s communication style. GPT-4O then drafts a single seed utterance (the turn-0 user message used in the on-policy phase) that is grounded in at least one element of E_t and worded in the persona’s voice. The seed is identical across all seven tested systems for the same (persona, session); divergence begins only at turn 1 when the system’s reply enters the simulator’s context.

Evaluation question generation (stage 5).

For each (persona, session), 10 questions are sampled to form the evaluation slate: 5 POS-fresh ask about facts newly exposed in session t ; 3 POS-tracking re-ask a fixed concept-level question (e.g. “which city do you live in?”) across multiple sessions with the gold answer flipping according to σ_t ; the remaining 2 are NEG queries whose gold response is to refuse to answer, drawn from a balanced mix of never-mentioned content, now-stale evidence after a state

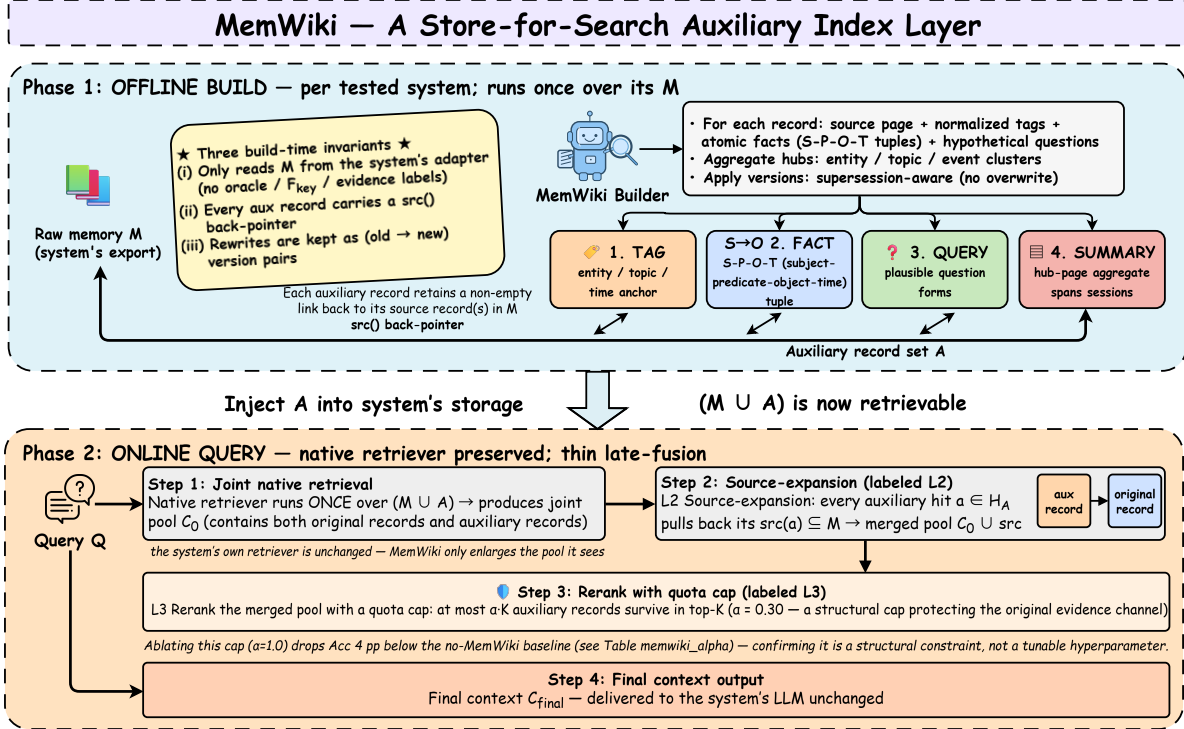


Figure 7: MemWiki as a store-for-search auxiliary layer. The raw memory $\mathcal{M}$ is organised into a wiki-style index whose source pages and hub pages are flattened into four AuxRecord classes (factual, tag, query-shaped, aggregate). The system’s native retriever runs over $\mathcal{M} \cup \mathcal{A}$; any auxiliary hit pulls in its source records via $src(\cdot)$, and a single rerank pass orders the merged pool subject to a 30% auxiliary-record quota.

flip, privacy-sensitive content, and under-specified queries. After offline deduplication and consistency filtering, the slate yields 962 asks (790 POS, 172 NEG); within the POS subset, 400 are state-tracking re-asks of ~ 100 unique concept questions. Each question stores its `ask_at_sessions` schedule, the per-session `gold_answer`, and, for state-tracking and stale-evidence items, the per-session `trap_answer` (the value the system would return if it failed to track the flip).

Oracle context generation (stage 6). Per (question, asked-session) pair, GPT-4O synthesises a minimal dialogue script grounded in σ_t and E_t , then extracts a short oracle context $\mathcal{C}_{oracle}$ that fully supports the gold answer. The oracle channel is independent of any tested system’s dialogue — it only feeds the Generation Examiner. Oracle contexts for NEG asks are empty.

On-policy session loop (per-system). For each (system, persona), we reset the memory store and iterate 10 sessions. Within a session, turn 0 injects the offline seed utterance; turns 1–18 are produced by a GPT-4O user simulator conditioned on the persona backstory, current state σ_t , residual must-expose list E_t^{rem} (initialised to E_t ,

decremented after each turn by `mark_exposed`), full dialogue history, and the system’s last reply. The user simulator returns a JSON object `{text, exposed_in_this_msg}`; only the `text` field is fed to the tested system, while `exposed_in_this_msg` drives `mark_exposed`. If turn 19 arrives with $E_t^{rem} \neq \emptyset$, a deterministic `force_expose` routine emits one final user turn that explicitly mentions every remaining (variable, value) pair in the persona’s voice; the tested system processes the force turn as a normal user message. Across the $7 \times 10 \times 10 = 700$ (system, persona, session) cells, `force-expose` fires on 11.6% of cells on average (4.1% on concise personas, up to 21.4% on indirect ones). End-of-session evaluation iterates over all questions whose `ask_at_sessions` contains t and routes each ask through a `READ-ONLYADAPTER` that blocks the system’s write hooks, so probe traffic never modifies $\mathcal{M}$.

Quality gates. Two gates govern dataset release. (i) *Offline auto-verification*: every persona’s offline bundle is checked against five construction constraints (schema–state consistency, must-expose coverage, POS oracle support, NEG unanswerability, trap/temporal consistency); failed artifacts are

System	TAG	FACT	QUERY	SUM.	Sum	All	Crowd loss
TiMem	+0.1	+0.4	+0.2	+0.1	+0.8	+0.3	−62.5%
O-Mem	+0.4	+0.9	+0.6	+0.5	+2.4	+1.5	−37.5%
EverMemOS	+0.2	+0.5	+0.4	+0.3	+1.4	+1.0	−28.6%
MemOS	+0.6	+1.5	+1.1	+0.8	+4.0	+2.6	−35.0%
MemBox	+1.1	+2.6	+1.7	+1.4	+6.8	+4.2	−38.2%
MemoryOS	+0.8	+1.8	+1.3	+1.0	+4.9	+3.0	−38.8%
GAM	+1.2	+2.7	+1.9	+1.5	+7.3	+4.6	−37.0%

Table 16: MemWiki AuxRecord single-class ablation (Δ Final Acc, pp). TAG=tag records, FACT=factual records, QUERY=query-shaped records, SUM.=aggregate summaries. **Sum**=sum of singletons; **All**=all four enabled; **Crowd loss**= $1 - \text{All}/\text{Sum}$, the share of gain dropped to the 30% auxiliary quota.

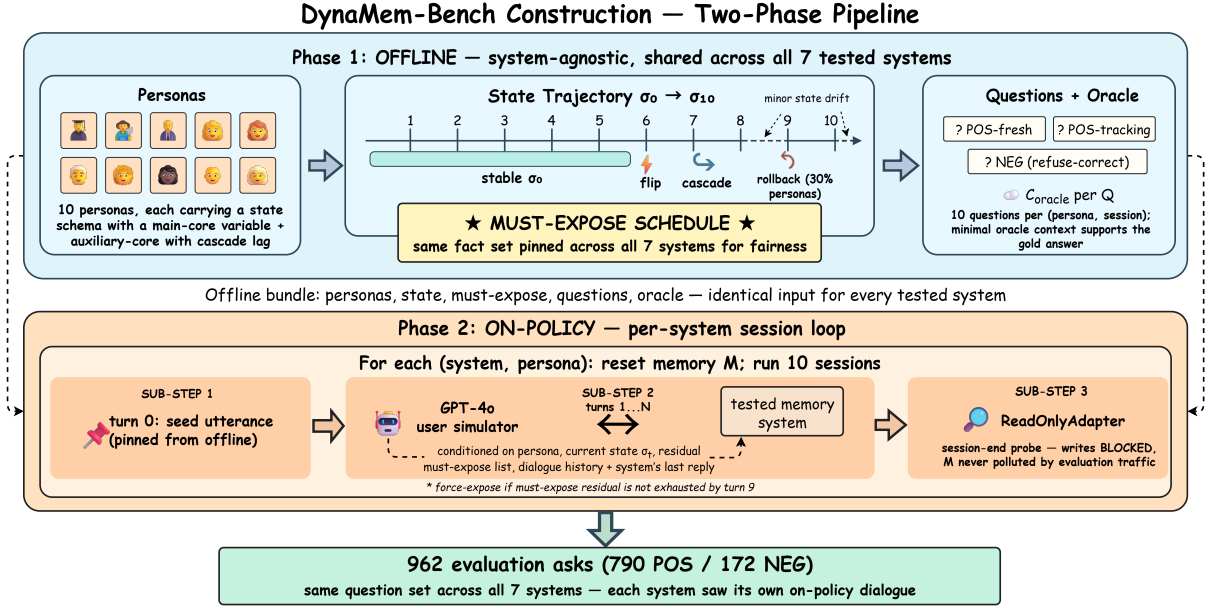


Figure 8: DynaMem-Bench construction. An offline phase fixes everything the cross-system comparison requires: 10 personas, a 10-session state schedule with cascade and rollback events, must-expose sets E_t , seed utterances, evaluation questions, and oracle contexts. An on-policy phase is repeated per system: a user simulator drives the dialogue conditioned on σ_t and residual exposure, with a force-expose fallback at turn 19. Probe questions go through a read-only adapter so they cannot pollute $\mathcal{M}$.

regenerated and checked again before release. (ii) *On-policy exposure audit*: after each (system, persona, session) run, an offline auditor reads the dialogue and re-checks that every item in E_t is actually exposed; the audit must pass at 100% before the session’s evaluation results are admitted. The GEMINI-2.5-PRO verifier used for offline auto-verification is separately audited against trained human annotators in Appendix D.2.

C.10 DynaMem-Bench Audit and Meta-Evaluation

Static vs. dynamic comparison. Table 23 reports the per-system accuracy delta from Locomo to DynaMem-Bench together with the mean per-layer defect-rate movement. Pure retrieval-driven systems (GAM −10.6, EverMemOS −11.4) de-

grade most. The accuracy drop is absorbed by retrieval (+3.8 pp on Ret) and encoding (+2.0 pp on Enc); generation defects *decrease* by −1.5 pp because the static Locomo questions include several multi-hop temporal items that are absent from DynaMem-Bench’s must-expose schedule.

Per-subset breakdown. Table 24 reports the accuracy on the four difficulty subsets of DynaMem-Bench (asks aggregated $\times 7$ systems). The rollback subset is the hardest at 47.3%, well below the static-state $\sim 61\%$ baseline; state-tracking re-asks (53.6%) confirm that consistent multi-session truth-tracking is unsolved across all seven systems. Immediate cascades (65.2%) are recovered better than delayed cascades (58.5%), suggesting that systems anchor on the original record’s session more

System	b	c	$c-b$	χ^2	p -value	sig.
TiMem	19	25	6	0.82	0.366	—
O-Mem	20	50	30	12.86	$< 10^{-3}$	***
EverMemOS	20	40	20	6.67	0.010	*
MemOS	28	80	52	25.04	$< 10^{-3}$	***
MemBox	47	130	83	38.92	$< 10^{-3}$	***
MemoryOS	38	98	60	26.47	$< 10^{-3}$	***
GAM	58	149	91	40.00	$< 10^{-3}$	***
Aggregate	230	572	342	145.84	$< 10^{-3}$	***

Table 17: Per-system McNemar tests for MemWiki on LoCoMo ($N = 1986$ paired observations per system). b = baseline correct $\wedge$ MemWiki wrong (*spoils*); c = baseline wrong $\wedge$ MemWiki correct (*recovers*); $\chi^2 = (b-c)^2 / (b+c)$ with $df = 1$. Symbols: *: $p < 0.05$; ***: $p < 10^{-3}$. Six of seven systems show significant improvement; TiMem’s marginal +0.3 pp gain is not significant, consistent with its −62.5% crowd loss (Table 16).

strongly than on the rewriting session.

Failure-recovery analysis. Table 25 isolates a stricter measurement: for each rewritten fact we partition its re-asks into pre-flip and post-flip windows. Pre-flip accuracy is comparable across systems (63.0% mean) because the question is essentially static recall. Post-flip accuracy diverges sharply (51.5% mean, −11.5 pp). Retrieval-driven systems (EverMemOS, GAM) suffer the largest drops because their stores append rather than supersede records, so a stale record continues to be retrieved even after the flip. TiMem and MemoryOS, which both maintain salience or recency-weighted indices, lose less because the rewritten record’s higher recency promotes it above the stale one.

Auto-verification. The offline auto-verification gate is described above; here we focus on the downstream dynamic effects after the released artifacts enter on-policy evaluation.

Force-expose frequency. Force-expose is invoked in 11.6% of (persona, session) pairs averaged across the seven systems. The frequency is highest on personas with verbose communication style and lowest on terse personas.

Reliability and meta-evaluation. Appendix D reports random-sampling reliability studies for the answer scorer, the three Examiners, and the GEMINI-2.5-PRO construction verifier.

System	Acc $\uparrow$		Ret $\downarrow$	
	Bsl	+ MemWiki	Δ	Δ
TiMem	75.3	75.6	+0.3	−0.9
O-Mem	74.9	76.4	+1.5	−2.3
EverMemOS	72.4	73.4	+1.0	−1.6
MemOS	70.1	72.7	+2.6	−3.0
MemBox	63.2	67.4	<u>+4.2</u>	<u>−5.0</u>
MemoryOS	60.8	63.8	+3.0	−4.0
GAM	60.2	64.8	+4.6	−5.8
mean	68.1	70.6	+2.5	−3.2

Table 18: MemWiki retrieval-side intervention on LoCoMo with GPT-4O-MINI. MemWiki is built per-system from each baseline’s own memory export, then re-injected through L1/L2/L3 ($\leq 30\%$ auxiliary quota). Final Acc (%) and retrieval defect rate Ret (%) before/after intervention. **Bold red**: largest gain; blue underline: second-largest. Aggregate McNemar over 7 systems $\times 1986$ paired observations: $\chi^2 = 145.8$ ($p < 10^{-3}$); per-system breakdown in Appendix C.7, Table 17.

C.11 User Simulator Robustness

DynaMem-Bench decouples the *world generator* (the must-expose schedule E_t , state evolution σ_t , and rollback subset, all produced by the offline pipeline) from the *renderer* (the user simulator LLM that verbalises the already-fixed facts in turns 1–18). To verify that this decoupling holds in practice, we re-run the full 7×962 matrix with QWEN3-30B-INSTRUCT replacing GPT-4o as the user simulator, while keeping every other variable fixed: the memory backbone (GPT-4O-MINI), the three Examiner judges, the embedding, top- K , all thresholds, the must-expose schedule, and the state machine. Only the LLM call in turns 1–18 changes; turn 0 (pinned seed) and turn 19 (force-expose) are pipeline-controlled and unaffected, and probe questions never go through the simulator.

Table 26 reports the result. Three robust findings follow. (i) Mean Acc shifts by only 0.1 pp ($60.8 \rightarrow 60.7\%$), and the three layer-aggregate rates are essentially unchanged ($|\Delta| \leq 0.1$ pp at the mean). (ii) Per-system $|\Delta \text{Acc}| \leq 1.1$ pp with mean 0.73 pp, well below the per-system standard deviation (≈ 7.6 pp). (iii) The ranking is preserved up to a single transposition between MemoryOS and MemBox, whose baseline accuracies differ by only 0.5 pp and which therefore sit within simulator-induced noise; Spearman $\rho = 0.964$, Pearson $r(\text{Acc}) = 0.995$. The same near-tie reordering pattern was reported by Jiayang et al. (2026) (Appendix F.2) when swapping user LLMs.

System	bsl	Final Acc (%) under MemWiki quota α						
		.10	.20	.30*	.40	.50	.70	1.0
TiMem	75.3	75.5	75.7	75.6	75.3	74.8	73.4	69.8
O-Mem	74.9	75.4	76.2	76.4	76.1	75.5	73.9	70.4
EverMemOS	72.4	72.9	73.3	73.4	73.5	72.8	70.6	67.5
MemOS	70.1	71.0	72.1	72.7	72.6	71.9	70.2	66.5
MemBox	63.2	64.7	66.3	67.4	67.5	66.8	64.6	60.5
MemoryOS	60.8	61.8	63.0	63.8	63.9	63.1	61.0	56.9
GAM	60.2	62.0	63.7	64.8	65.0	64.1	61.9	57.3
mean	68.1	69.0	70.0	70.6	70.6	69.9	67.9	64.1

Table 19: MemWiki α -quota ablation on LOCOMO (gpt-4o-mini). α caps the fraction of MemWiki auxiliary records in the top- $K = 10$ context, so αK is the maximum number of MemWiki slots ($\alpha = .30$ allows ≤ 3 , $\alpha = 1.0$ removes the cap). (*) $\alpha = 0.30$ is the default used in all main experiments. **Bold red**: per-system peak. Per-system optima span $\alpha \in \{0.20, 0.30, 0.40\}$, yet every system at $\alpha = 0.30$ is within 0.2 pp of its own peak. The plateau region $\alpha \in [0.30, 0.40]$ gives identical mean Final Acc (70.6%); removing the quota ($\alpha = 1.0$) drops mean Final Acc to 64.1%, 4.0 pp below the no-MemWiki baseline.

Magnitude comparison. The simulator-induced mean $|\Delta \text{Acc}|$ (0.73 pp) is roughly 1/5 of the backbone shift (GPT-4O-MINI $\rightarrow$ GPT-4.1, 3.7 pp) and 1/10 of the dataset shift (LOCOMO $\rightarrow$ DynaMem-Bench, 7.3 pp); the simulator is the smallest of the variation sources we measured. The cross-system gap (TiMem 71.2% vs. GAM 49.6%, 21.6 pp) is therefore not driven by simulator choice.

D Reliability and Dataset-Quality Validation

D.1 LLM-as-Judge Reliability

Validation question. EvalMem uses model-based judgments at two points, and each requires a separate reliability check (Jung et al., 2025; Wang et al., 2024b; Zhu et al., 2025). In *answer scoring*, a judge maps a query, the gold answer or expected refusal, and the system response to a binary CORRECT/WRONG label. In *diagnostic attribution*, the Encoding, Retrieval, and Generation Examiners inspect operation-specific evidence and assign local states, which are then mapped to defect codes (Appendix B.3). These two decisions have different ambiguity sources: answer scoring is mainly sensitive to paraphrases and non-canonical refusals, whereas Examiner scoring must distinguish partial memory support, topically related but insufficient retrieval, and generation failures near the GF/GRF boundary. We therefore validate answer-

level scoring and Examiner-level attribution separately against trained human annotators.

Annotation protocol. All audit samples in this subsection are blinded to system identity, dataset identity, and the LLM judge output. Three trained annotators — master’s-level researchers in NLP and long-term memory evaluation — first complete a 30-example calibration round that is excluded from the reported scores. For each audited item, two annotators label independently and the third annotator adjudicates disagreements. For binary answer correctness, annotators label CORRECT/WRONG; for Examiner validation, annotators label the Examiner state and the induced defect set using the same definitions as Appendix B.3. Audit samples are drawn by uniform random sampling from completed evaluation logs. We report raw agreement, Cohen’s κ , and, for Examiner outputs, micro-F1 over defect sets.

Answer-level judge validation. We draw a uniform random sample of 120 system-question pairs from the pooled evaluation logs across LOCOMO, LONGMEMEVAL-S, and DynaMem-Bench. The decision input contains only the query, the gold answer or expected refusal, and the system response. Semantically equivalent paraphrases of the gold answer are marked correct; for unanswerable questions, only explicit refusal or a clear acknowledgment of insufficient evidence is marked correct. The main answer judge is gpt-5-mini. Table 27 reports its agreement with the adjudicated human label.

Answer-level analysis. The answer judge reaches 96.7% agreement with adjudicated human labels and $\kappa = 0.91$. The four LLM-human disagreements cluster in three narrow boundary cases: (i) over-long answers that contain both the correct value and irrelevant stale details, (ii) unanswerable-case responses that hedge before giving a concrete guess, and (iii) temporal answers whose surface form differs from the gold but denotes the same interval. As a cross-judge robustness check, we re-score the seven-system LOCOMO run with a backup gpt-4o judge: mean Final Acc shifts by at most 0.6 pp and no system ranking changes, with the largest per-system shift on MemoryOS (+0.6 pp) consistent with that system producing longer free-form answers closer to the judge boundary.

System	Metric	$K=1$	$K=2$	$K=3$	$K=5$	$K=8$	$K=10$	$K=12$	$K=15$	$K=20$
TiMem	Hit@ K	38.4	47.5	55.7	64.7	71.0	73.5	75.2	77.3	79.5
	Final Acc	64.2	68.5	70.5	73.6	75.0	75.3	75.5	75.3	74.5
O-Mem	Hit@ K	36.4	45.5	54.3	63.2	68.7	71.1	73.0	75.2	77.7
	Final Acc	63.8	68.0	70.2	73.0	74.7	74.9	74.8	74.0	72.6
EverMemOS	Hit@ K	34.9	43.8	52.8	61.8	67.2	69.4	71.4	73.5	76.0
	Final Acc	61.6	65.8	67.9	70.5	72.1	72.4	72.3	71.6	70.0
MemOS	Hit@ K	33.7	42.4	50.7	59.7	64.8	66.7	68.7	71.0	73.3
	Final Acc	59.7	63.7	65.9	68.5	70.4	70.1	69.8	69.0	67.7
MemBox	Hit@ K	30.0	38.0	45.8	53.3	58.5	60.3	62.4	64.9	67.3
	Final Acc	53.9	57.5	59.8	62.4	63.7	63.2	62.8	62.0	60.5
MemoryOS	Hit@ K	28.7	36.4	44.0	51.6	56.5	58.6	60.6	63.0	65.3
	Final Acc	51.4	55.2	57.0	61.0	61.1	60.8	60.4	59.5	58.0
GAM	Hit@ K	27.9	35.5	42.6	50.0	55.2	57.0	59.0	61.3	63.6
	Final Acc	50.8	54.5	56.6	58.7	60.0	60.2	60.0	59.2	57.7
<i>mean</i>	Hit@ K	32.9	41.3	49.4	57.8	63.1	65.2	67.2	69.5	71.8
	Final Acc	57.9	61.9	64.0	66.8	68.1	68.1	67.9	67.2	65.9

Table 20: Combined top- K sensitivity on LoCoMo with GPT-4O-MINI. **Hit@ K** (% , POS subset) is monotone increasing in K , whereas **Final Acc** (%) peaks at $K \in \{8, 10\}$ for every system. **Red red** marks each row’s Final Acc peak; the two yellow columns mark the $K \in \{8, 10\}$ sweet spot shared by Hit@ K ’s elbow and Acc’s maximum. Beyond this band, Acc declines although Hit@ K keeps rising; the gap is explained by the NOI rate in Table 21. System rankings under both metrics are stable across all K (Spearman $\rho \geq 0.96$ between any two columns).

K	RF	LATE	NOI	Ret. total
1	16.1	0.0	0.2	16.3
2	14.4	0.3	0.5	15.2
3	12.5	0.7	1.0	14.2
5	9.8	1.4	2.1	13.3
8	6.6	2.9	3.7	13.2
10	5.0	3.4	5.5	13.9
12	4.2	3.6	7.2	15.0
15	3.3	3.7	9.4	16.4
20	2.6	3.7	12.5	18.8

Table 21: Retrieval-layer code rates (%) on LoCoMo POS as a function of top- K (mean over 7 systems). **RF** (retrieval failure: key fact never recalled) decays monotonically as more candidates are fetched. **LATE** (key fact recalled below rank-5) and **NOI** (low signal-to-noise ratio) grow with K . The total retrieval defect rate is U-shaped, with a flat minimum at $K \in \{8, 10\}$ — the same range that maximises Final Acc in Table 20.

Examiner-level judge validation. For Examiner validation we randomly sample 150 questions from the completed evaluation logs and audit each Examiner’s decision on those same questions. For each sampled question, annotators relabel the state and induced defect set for the Encoding, Retrieval, and Generation Examiners, yielding 150 audited decisions per Examiner and 450 decisions in total. Annotators receive only the evidence available to the relevant Examiner: the memory-store candidate pool for Encoding, the native top- K retrieval list

Session	Rule
1–4	Initial state σ_0 (introduce baseline facts)
5	Hold σ_4 (let old facts settle)
6	Main-core change (e.g. city: SH→BJ)
7	Auxiliary cascade (e.g. commute)
8	ρ -driven single-point update
9	Rollback for 30% of personas
10	ρ -driven update + terminal stable state

Table 22: State trajectory rules used to evolve σ_t across sessions in DynaMem-Bench; cascade and rollback target dynamic-state failures.

with rank/SNR diagnostics for Retrieval, and the online/oracle/gold answer triple for Generation. Table 28 reports state agreement and micro-F1 over the induced defect-code set.

Examiner-level analysis. All three Examiner judges reach $\kappa \geq 0.86$ against adjudicated human labels, with micro-F1 between 0.92 and 0.95 over the induced defect codes. Retrieval is the most stable because the key decision is grounded in observable top- K evidence and rank/SNR diagnostics. Generation is slightly harder because the GF/GRF boundary requires distinguishing whether the model ignored oracle evidence or used it but reasoned incorrectly. Encoding disagreements mostly involve compressed summaries that preserve the answer but omit explicit entity or temporal anchors; the released prompts and attribution matrix (Ap-

System	LoCoMo10	DynaMem-Bench	Δ (pp)
TiMem	75.3	71.2	-4.1
O-Mem	74.9	68.9	-6.0
MemOS	70.1	62.5	-7.6
EverMemOS	72.4	61.0	-11.4
MemoryOS	60.8	56.4	-4.4
MemBox	63.2	55.9	-7.3
GAM	60.2	49.6	-10.6
mean	68.1	60.8	-7.3
<i>Per-layer defect rate (mean over 7 systems)</i>			
Encoding	7.7	9.7	+2.0
Retrieval	22.1	25.9	+3.8
Generation	6.5	5.0	-1.5

Table 23: **Static vs. dynamic comparison.** The mean accuracy drop (-7.3 pp) is absorbed by retrieval (+3.8 pp) and encoding (+2.0 pp); generation defects *decrease* (-1.5 pp), reflecting that DynaMem-Bench has fewer multi-hop temporal questions than LoCoMo. Pure retrieval-driven systems (GAM -10.6, EverMemOS -11.4) degrade most.

Difficulty subset	N (asks)	Correct	Acc
state-tracking (multi-session re-asks)	2800	1500	53.6
immediate cascade (3 personas)	609	397	65.2
delayed cascade (7 personas)	1421	831	58.5
rollback (3 personas, sess. 9)	602	285	47.3

Table 24: **Per-subset accuracy on DynaMem-Bench (7 systems, $\times 7$ aggregation).** Rollback is hardest: half the answers go to the wrong direction of the state flip. State-tracking (53.6%) shows that consistent multi-session truth-tracking is unsolved.

pendix B.3) make these residual boundary decisions auditable.

D.2 Gemini Verifier Reliability for DynaMem-Bench

Validation question. DynaMem-Bench uses LLMs in two roles during construction (Appendix C.9): GPT-4O drafts personas, state schedules, exposure plans, questions, and oracle contexts, while GEMINI-2.5-PRO independently verifies whether these artifacts satisfy the construction constraints. The reliability question is therefore whether Gemini’s construction-time pass/fail decisions match expert quality-control labels. We answer this by comparing Gemini verifier verdicts with adjudicated human annotations on randomly sampled construction-log items.

Protocol. For each of the five verifier checks (schema-state consistency, must-expose coverage, POS oracle support, NEG unanswerability,

System	Pre-flip	Post-flip	Δ
TiMem	73.4	62.1	-11.3
O-Mem	71.0	60.7	-10.3
MemOS	64.8	53.5	-11.3
EverMemOS	63.1	45.5	-17.6
MemoryOS	58.7	52.6	-6.1
MemBox	57.9	47.4	-10.5
GAM	52.1	38.6	-13.5
mean	63.0	51.5	-11.5

Table 25: Failure-recovery analysis on DynaMem-Bench. For each persona we identify session τ at which a fact is rewritten (cascade or rollback) and split that fact’s re-asks into the **pre-flip** window (asked before τ , ground truth = original) and the **post-flip** window (asked after τ , ground truth = revised). Accuracy in the post-flip window measures the system’s ability to overwrite stale evidence. Retrieval-driven systems (EverMemOS, GAM) suffer the largest drops because their stores append rather than supersede records.

trap/temporal consistency), we draw a uniform random sample of Gemini’s construction-time verdicts; the sample contains both pass and fail decisions (overall fail rate $\approx 18\%$ across the five checks). Each item is relabeled under the same binary criterion used by the verifier: PASS if the artifact satisfies the target construction constraint, and FAIL otherwise. Two trained annotators label each item independently, and a third annotator adjudicates disagreements. Annotators see the artifact and the hidden state or oracle evidence required for the decision, but neither the Gemini verdict nor the builder prompt.

Analysis. GEMINI-2.5-PRO reaches 97.0% overall agreement and $\kappa = 0.89$ with adjudicated human labels, close to the human-human ceiling (97.9%, $\kappa = 0.92$). The gap is concentrated on NEG unanswerability and trap/temporal consistency, where stale-but-related evidence and close paraphrases of the current value sit near the decision boundary; on the more anchorable checks (schema and must-expose) the verifier is within 1.3 pp of the human ceiling. The result supports using GEMINI-2.5-PRO as the construction-time quality gate for DynaMem-Bench.

E Case Study and Examiner Prompts

This appendix grounds the framework in a single end-to-end case (§E.1) and lists the seven prompts that implement the three Examiners and the Attribution Agent (§E.2). The case is intentionally a RETRIEVAL-only failure so that the contrast among en-

System	Acc		Enc		Ret		Gen	
	GPT-4o	Qwen3	GPT-4o	Qwen3	GPT-4o	Qwen3	GPT-4o	Qwen3
TiMem	71.2	70.3	7.6	7.9	19.0	19.4	4.3	4.4
O-Mem	68.9	69.6	7.5	7.3	21.5	21.1	4.5	4.4
MemOS	62.5	61.4	9.0	9.4	25.4	25.9	4.9	5.1
EverMemOS	61.0	60.6	9.6	9.7	26.1	26.4	4.7	4.8
MemoryOS	56.4	55.8	11.0	11.2	28.4	28.8	5.1	5.2
MemBox	55.9	56.7	10.0	9.8	29.1	28.6	5.5	5.4
GAM	49.6	50.2	13.0	12.8	32.0	31.7	5.9	5.8
<i>mean</i>	60.8	60.7	9.7	9.7	25.9	26.0	5.0	5.0

Table 26: DynaMem-Bench under two user simulators (GPT-4o vs. QWEN3-30B-INSTRUCT), with the memory backbone fixed at GPT-4O-MINI and the must-expose schedule and state evolution preserved. Mean accuracy shifts by 0.1 pp; per-system $|\Delta\text{Acc}| \leq 1.1$ pp (mean 0.73 pp). The ranking is preserved up to a single transposition between MemoryOS and MemBox, whose baseline accuracies differ by only 0.5 pp; Spearman $\rho = 0.964$, Pearson $r = 0.995$.

Judge	N	LLM-human agr.	Cohen’s κ
Answer judge	120	96.7%	0.91

Table 27: Answer-level judge validation on 120 randomly sampled system-question pairs. Agreement and Cohen’s κ compare the main answer judge with adjudicated human labels; no human-human ceiling is assumed.

coding, retrieval, and generation verdicts is visible. Following the comparison schema in Appendix B.3, the Generation Examiner judges the online answer and the oracle answer side-by-side, which lets the Attribution Agent rule out hidden generation faults.

E.1 Case Walk-Through: conv-26:94

Sample. The query is conv-26:94 from LO-CoMo, a single-hop POS question: “What is Melanie’s hand-painted bowl a reminder of?”. The gold answer is “art and self-expression”; the evidence is a single turn (D4:5) where Caroline says “like my hand-painted bowl. . . it reminds me of art and self-expression”. MEMBOX (the tested system here) answers “The meaning is not specified.” end-to-end and is scored wrong; if the same backbone is given D4:5 directly as oracle context it answers “art and self-expression” and is scored correct.

Encoding Examiner. The adapted AgenticRAG routine runs three rounds over MEMBOX’s exported store and surfaces ctx-0, a stored fragment of D4:5, at rank 1. The Examiner returns encoding_state = Exist with no defect: the bowl-meaning fact is physically present in the store.

Retrieval Examiner. MEMBOX’s native retriever returns a top-10 dominated by topic-

adjacent records (Melanie’s sunrise painting, colour discussions, “what does the piece mean” from a different art thread); the bowl fragment ctx-0 is absent (rule-based diagnostics return rank = -1, hit indices = []). The Examiner returns retrieval_state = Miss, defect set {RF}. LATE and NOI are not computable because no hit exists.

Generation Examiner. The Examiner is given the online answer (“The meaning is not specified.”), the oracle answer (“art and self-expression”), and the gold. It returns online_correct = false, oracle_correct = true, generation_state = PASS, defect set $\emptyset$. The comparative_judgement.online_vs_oracle field explicitly attributes the online failure to the upstream pipeline: “the backbone produces the correct answer once D4:5 is provided directly, so the online-side failure is fully attributable to the upstream retrieval miss”.

Attribution. The Attribution Agent unions the three verdicts. The encoding suppression rule (§2.2) does not fire because $S_{\text{enc}} = \text{EXIST}$, so RF is preserved. The final attribution is $\mathcal{D}_{\text{total}} = \{\text{RF}\}$ with primary_cause = retrieval. The Attribution Agent’s decision_trace reuses the Generation Examiner’s online_vs_oracle field as the auditable reason why the Generation layer is absolved.

Why this case illustrates the value of multi-label attribution. End-to-end scoring records conv-26:94 as “MEMBOX wrong”. EvalMem instead localises the failure to retrieval only and rules out encoding and generation with separate evidence. The case is also one of the 130 MEMBOX questions recovered by MemWiki (Table 18),

Judge target	#Cases	Human–human agr./ κ	LLM–human agr./ κ	Defect F1
Encoding Examiner	150	96.0/0.91	94.7/0.89	0.94
Retrieval Examiner	150	96.7/0.92	95.3/0.90	0.95
Generation Examiner	150	94.7/0.88	93.3/0.86	0.92
All Examiners	450	95.8/0.90	94.4/0.88	0.94

Table 28: Examiner-level judge validation on 150 randomly sampled questions, yielding 450 audited Examiner decisions. Human–human reports agreement between the two independent annotators; LLM–human compares the main Examiner output with adjudicated human labels; Defect F1 is computed over the induced defect-code set.

Verification target	Random sample	Human–human agr./ κ	Gemini–human agr./ κ
Schema–state consistency	80 bundles	98.8/0.95	97.5/0.92
Must-expose coverage	200 exposure events	99.0/0.96	98.5/0.94
POS oracle support	200 POS asks	97.5/0.91	96.5/0.88
NEG unanswerability	120 NEG asks	96.7/0.89	95.8/0.86
Trap / temporal consistency	120 tracking asks	97.5/0.91	96.7/0.88
Overall	720 checks	97.9/0.92	97.0/0.89

Table 29: Reliability of GEMINI-2.5-PRO as the DynaMem-Bench construction verifier against adjudicated human labels. The audit sample is drawn by uniform random sampling and includes both pass and fail verdicts overall fail rate $\approx 18\%$.

which is exactly the kind of failure MemWiki is designed to fix: the evidence is in the store, but the native retriever does not surface it.

E.2 Examiner Prompts

The seven prompts below implement the three Examiners and the Attribution Agent verbatim from `eval_core/prompts.py`. All prompts share the same output convention: output exactly one JSON object, never wrapped in Markdown, with defect codes chosen only from the enumeration specified by the prompt.

Encoding Examiner — POS.

You are the EncodingAgent. For a POS query, judge whether the memory system has written the information required to answer the question into its observable memory store.

Variable definitions:

- Query: the current question.
- F_key: the minimal set of fact units that must be supported.
- GoldEvidence: ground-truth evidence (semantic reference; NOT exact-match requirement).
- Candidates: the memory system's exportable candidate records. Judge ONLY on these candidates; do not hallucinate hidden memory.

State definitions:

- EXIST: at least one candidate semantically supports the core fact.
- MISS: no candidate supports the core fact (POS encoding failure).
- CORRUPT_AMBIG: a candidate covers the topic but subject / pronoun / time anchor / event attribution is too ambiguous to confirm the target fact.
- CORRUPT_WRONG: a candidate covers the topic but the key value (person, time, place, relation,

event outcome) is wrong.

Strict defect-code mapping:

- MISS $\rightarrow$ defects=["EM"] (Encoding Missing)
- CORRUPT_AMBIG $\rightarrow$ defects=["EA"] (Encoding Ambiguous)
- CORRUPT_WRONG $\rightarrow$ defects=["EW"] (Encoding Wrong)
- EXIST $\rightarrow$ defects=[]

Judging principles:

1. Use semantic equivalence, NOT exact string matching.
2. Summaries, paraphrases, time normalisation, pronoun resolution, and joint support across multiple candidates are all acceptable.
3. A candidate that is a summary still counts as EXIST if it preserves the core fact needed to answer Q.
4. Emit reasoning, matched_candidate_ids, evidence_snippets, and missing_facts for downstream attribution.

Output ONE JSON object conforming exactly to:

```
{
  "encoding_state":
  "EXIST|MISS|CORRUPT_AMBIG|CORRUPT_WRONG",
  "defects": ["EM|EA|EW"],
  "confidence": 0.0,
  "matched_candidate_ids": [],
  "reasoning": "...",
  "evidence_snippets": [],
  "missing_facts": []
}
```

```
Query: {query}
F_key: {f_key}
GoldEvidence: {evidence_texts}
Candidates:
{candidates_listing}
```

Encoding Examiner — NEG.

You are the EncodingAgent. For a NEG query, judge whether the memory system has written pseudo-facts that could mislead the generator into fabricating a concrete answer.

IMPORTANT: For NEG queries, MISS is the CORRECT state. It means the store contains NO pseudo-evidence usable to answer Q.

Variable definitions:

- Query: the current question.
- ReferenceEvidenceForContrast: contrast evidence explaining why the question should not be answered.
- Candidates: the memory system's exportable records.

State definitions:

- MISS: correct. No pseudo-fact in the store would support a concrete answer.
- DIRTY: wrong. The store contains pseudo-facts that would induce a concrete answer, including:
 - (i) over-generalisation of raw facts ("grandma" -> "grandparents");
 - (ii) cross-record synthesis fabricating new entities or relations;
 - (iii) speculative content written as assertion.

Strict defect-code mapping:

- DIRTY -> defects=["DMP"] (Dirty Memory Pollution)
- MISS -> defects=[]

Judging principles:

1. Mark DIRTY only when a candidate genuinely supplies a pseudo-fact / pseudo-time / pseudo-event association sufficient to drive a concrete (and wrong) answer.
2. Topical or lexical similarity alone -- without sufficient support for the wrong answer -- is still MISS.
3. System-induced semantic drift that exactly supports the wrong answer DOES count as DIRTY.
4. Emit reasoning and evidence_snippets explaining the verdict.

Output ONE JSON object conforming exactly to:

```
{
  "encoding_state":      "DIRTY|MISS",
  "defects":             ["DMP"],
  "confidence":          0.0,
  "matched_candidate_ids": [],
  "reasoning":           "...",
  "evidence_snippets":   []
}
```

```
Query: {query}
ReferenceEvidenceForContrast: {evidence_texts}
Candidates:
{candidates_listing}
```

Retrieval Examiner — POS.

You are the RetrievalAgent. For a POS query, judge whether the memory system's native retriever has surfaced evidence sufficient to support F_key into the top-K context.

Variable definitions:

- Query: the current question.
- F_key: the minimal fact units the question depends on.
- GoldEvidence: ground-truth evidence (semantic reference).
- RetrievedItems: the system's actual top-K output. Judge ONLY on these items.
- Diagnostics (advisory, NOT hard rules):
 - * rank_index: best-hit position estimated by rule-matching; -1 = not found.
 - * hit_indices: positions rule-matching considers possible hits.
 - * snr: estimated signal-to-noise ratio of the top-K context.
 - * tau_rank / tau_snr: thresholds for LATE / NOI flags.

State definitions:

- HIT: RetrievedItems contain evidence sufficient to support the core fact.
- MISS: RetrievedItems do NOT.

Strict defect-code mapping:

- RF = Retrieval Failure. Output ONLY when retrieval_state = MISS.

Post-processing will suppress RF if upstream encoding = MISS.

- LATE = Evidence is in top-K but at a late position

(best_rank > tau_rank).

- NOI = Evidence is in top-K but drowned by noise (snr < tau_snr).

Judging principles:

1. Semantic support; exact-match wording NOT required.
2. Summaries, compressed memory, paraphrasing, and time normalisation all constitute valid hits.
3. Joint support across multiple items also yields HIT.
4. On MISS, output RF. On HIT, append LATE / NOI as applicable (may co-occur).
5. On MISS (hit_indices = []), LATE / NOI sub-states are not computable; the defect set cleanly resolves to {RF}.
6. Emit best_rank, reasoning, and evidence_snippets for downstream attribution.

Output ONE JSON object conforming exactly to:

```
{
  "retrieval_state":    "HIT|MISS",
  "defects":            ["RF|LATE|NOI"],
  "matched_ids":        [],
  "best_rank":          -1,
  "confidence":         0.0,
  "reasoning":          "...",
  "evidence_snippets": []
}
```

```
Query: {query}
F_key: {f_key}
GoldEvidence: {evidence_texts}
Diagnostics: rank_index={rank_index},
hit_indices={hit_indices}, snr={snr:.6f},
tau_rank={tau_rank}, tau_snr={tau_snr}
RetrievedItems:
{retrieved_items_listing}
```

Retrieval Examiner — NEG.

You are the RetrievalAgent. For a NEG query, judge whether the memory system's native retriever has produced highly-misleading pseudo-relevant hits.

IMPORTANT: For NEG queries, MISS is the CORRECT state. It means the retrieval output is NOT sufficient to mislead the generator.

State definitions:

- MISS: correct. No retrieval hit can mislead the generator.
- NOISE: wrong. Some retrieval hit will significantly induce a concrete but unsupported answer.

Strict defect-code mapping:

- NOISE -> defects=["NIR"] (Noise-Induced Retrieval)
- MISS -> defects=[]

Judging principles:

1. Topical similarity alone is NOT NOISE. Only flag NOISE when the hit would push the generator toward a specific wrong answer.
2. Weak, generic, or background relevance is still MISS.
3. Any candidate flagged DMP by upstream Encoding SHOULD be considered NOISE if it appears at the top of native retrieval.
4. Emit reasoning and evidence_snippets explaining the verdict.

Output ONE JSON object conforming exactly to:

```
{
  "retrieval_state":    "MISS|NOISE",
  "defects":            ["NIR"],
  "confidence":         0.0,
  "reasoning":          "...",
  "evidence_snippets": []
}
```

```
Query: {query}
RetrievedItems:
{retrieved_items_listing}
```

Generation Examiner — POS.

You are the GenerationAgent for POS triple-answer comparison (online / oracle / gold).

Goal: separate "the error lives in the upstream pipeline (retrieval / encoding)" from "the error lives in generation itself".

Variable definitions:

- OnlineAnswer: the system's response under the actual RetrievedContext (end-to-end answer).
- OracleAnswer: the system's response under OracleContext.
- GoldAnswer: the reference answer.
- OracleContext: the perfect-evidence context (minimal subset supporting F_key).

Strict defect-code mapping:

- GF = Generation Faithfulness failure: ignores / contradicts OracleContext, relies on parametric knowledge instead.
- GRF = Generation Reasoning Failure: reads OracleContext but breaks multi-hop chain / temporal arithmetic / aggregation.
- When generation_state = PASS, defects = [].

Judging principles:

1. First judge online vs. gold and oracle vs. gold for semantic equivalence on the core fact; fill online_correct and oracle_correct.
2. If OracleAnswer is correct but OnlineAnswer is wrong, the failure lies in the upstream pipeline (retrieval or context pollution); generation_state = PASS, defects = []. The conclusion MUST be explicitly written into comparative_judgement.online_vs_oracle.
3. If OracleAnswer is itself wrong:
 - If it ignores or contradicts OracleContext -> prefer GF;
 - If it cites OracleContext but reasoning / comparison / induction fails -> prefer GRF;
 - generation_state = FAIL.
4. If both OracleAnswer and OnlineAnswer are correct, generation_state = PASS, defects = [].
5. The three fields in comparative_judgement must each be one sentence for reviewer auditability.
6. Different time-format expressions that point to the same time are equivalent ("the week before 6 July 2023" == "29 June - 5 July 2023").

Output ONE JSON object conforming exactly to:

```
{
  "generation_state": "PASS|FAIL",
  "defects": ["GF|GRF"],
  "online_correct": true,
  "oracle_correct": true,
  "comparative_judgement": {
    "online_vs_gold": "...",
    "oracle_vs_gold": "...",
    "online_vs_oracle": "..."
  },
  "reasoning": "..."
}
```

```
Query: {query}
GoldAnswer: {answer_gold}
OnlineAnswer: {answer_online}
OracleAnswer: {answer_oracle}
OracleContext: {oracle_context}
```

Generation Examiner — NEG.

You are the GenerationAgent for NEG triple-answer comparison.

NOTE: the correct behaviour for NEG is refusal (explicit insufficiency acknowledgement). OracleContext is empty; ExpectedRefusal specifies the canonical refusal style.

Strict defect-code mapping:

- GH = Generation Hallucination
- When generation_state = PASS, defects = [].

Judging principles:

1. First judge whether OnlineAnswer and OracleAnswer each constitute a valid refusal against ExpectedRefusal; fill online_correct and

oracle_correct.

2. If EITHER OnlineAnswer or OracleAnswer fabricates concrete content, generation_state = FAIL and defects = ["GH"].
3. If BOTH answers are cautious refusals or explicit insufficiency acknowledgements, generation_state = PASS, defects = [].
4. Key pattern: when OracleAnswer refuses but OnlineAnswer fails, this is the typical "upstream pollution overrides backbone refusal capability" -- still record GH (generation ultimately failed to refuse), but comparative_judgement.online_vs_oracle MUST explicitly state that the root cause sits in the retrieval side so that AttributionAgent can subsequently union NIR / DMP into D_total.
5. Each of the three fields in comparative_judgement must be one sentence.

Output ONE JSON object conforming exactly to:

```
{
  "generation_state": "PASS|FAIL",
  "defects": ["GH"],
  "online_correct": true,
  "oracle_correct": true,
  "comparative_judgement": {
    "online_vs_gold": "...",
    "oracle_vs_gold": "...",
    "online_vs_oracle": "..."
  },
  "reasoning": "..."
}
```

```
Query: {query}
ExpectedRefusal: {answer_gold}
OnlineAnswer: {answer_online}
OracleAnswer: {answer_oracle}
OracleContext: {oracle_context}
```

Attribution Agent.

You are the AttributionAgent. Combine the encoding / retrieval / generation probe outputs into the final defect set and the explanatory attribution.

NOTE: the final attribution is NOT a single root cause but a UNION of defects. One sample may carry multiple defects simultaneously.

Inputs:

- TaskDefinition: POS- or NEG-specific instruction.
- Query / GoldAnswer: context for review.
- EncodingSummary / RetrievalSummary / GenerationSummary: full JSON outputs from the three Examiners. GenerationSummary uses the comparison schema and carries online_correct / oracle_correct / comparative_judgement (three fields), which directly explains why Gen layer is or is not responsible.

Judging principles:

1. Encoding answers "was it stored?". Retrieval answers "was it surfaced?". Generation answers "given perfect evidence, could the model answer?".
2. Base the candidate attribution on the three probe outputs themselves, not on final correctness alone; after this union, apply the final-answer error gate for reported defect statistics, retaining codes only for incorrectly answered online queries.
3. If encoding_state = MISS, suppress RF: when not stored, retrieval failure should not be attributed.
4. For POS, encoding MISS is a failure. For NEG, encoding MISS is the correct observation and must NOT be auto-flagged as a defect.
5. For POS retrieval_state = MISS, RF is a candidate defect; keep it only after reconciling with encoding state.
6. Generation defects (GH / GF / GRF) may co-occur with upstream defects; never collapse into a single root cause.
7. primary_cause names the most responsible layer (encoding / retrieval / generation / none); defect_union MUST preserve ALL evidence-supported

defects.
8. secondary_causes must come from probe evidence; do not fabricate.
9. decision_trace must contain at least 4 entries: 3 per-layer verdicts plus 1 suppression / consistency check. When GenerationSummary.generation_state = PASS and online_correct = false, decision_trace MUST cite comparative_judgement.online_vs_oracle to explain why generation is absolved.

Output ONE JSON object conforming exactly to:

```
{
  "primary_cause": "encoding|retrieval|generation|none",
  "secondary_causes": [],
  "defect_union": [],
  "decision_trace": [],
  "summary": "..."}
}
```

TaskDefinition: {pos_or_neg_description}

Query: {query}

GoldAnswer: {answer_gold}

EncodingSummary: {enc_summary}

RetrievalSummary: {ret_summary}

GenerationSummary: {gen_summary}